



Universitat
de les Illes Balears

TRABAJO DE FIN DE GRADO

SPORTS LEAGUE: DISEÑO E IMPLEMENTACIÓN DE UN SISTEMA PARA LA GESTIÓN DE LIGAS DEPORTIVAS

Carlos Gálvez Mena

Grau d'Enginyeria Informàtica

Escola Politècnica Superior

Año académico 2025-26

SPORTS LEAGUE: DISEÑO E IMPLEMENTACIÓN DE UN SISTEMA PARA LA GESTIÓN DE LIGAS DEPORTIVAS

Carlos Gálvez Mena

Trabajo de Fin de Grado

Escola Politècnica Superior

Universitat de les Illes Balears

Año académico 2025-26

Paraules clau del treball: Liga deportiva, Aplicación multiplataforma, Spring Boot, React Native, TFG

Tutor: Antonio Crespí Tobeña

Gracias a todos los profesores que me han ayudado a lo largo de mi carrera, y en especial a mi primer tutor Raimon Massanet Vila por haber confiado en mi idea y, mi segundo tutor, Antonio Crespí Tobeña por su apoyo y orientación durante la realización de este trabajo, aun habiendo tenido que coger el relevo cuando el primer tutor tuvo que ser relevado por motivos burocráticos.

ÍNDICE GENERAL

Índice general	iii
Acrònims	vii
Resumen	ix
Resum	xi
1 Introducción	1
1.1 Alternativas actuales	2
1.1.1 Winner	2
1.1.2 Clupik	3
1.2 Motivación y objetivos	6
1.3 Estructura del documento	6
2 Planificación del proyecto	7
2.1 Metodología	7
2.2 Entregables	8
2.3 Planificación temporal	8
2.4 Hitos principales	9
2.5 Gestión de riesgos	9
3 Análisis y diseño del sistema	11
3.1 Especificación	11
3.1.1 Análisis de los interesados	11
3.1.2 Recogida de requisitos	13
3.1.3 Análisis de las respuestas de los formularios	19
3.1.4 Reunión con la OUSIS	25
3.1.5 Requisitos funcionales	26
3.2 Diseño	38
3.2.1 Arquitectura del sistema	38
3.2.2 Comportamiento del sistema	39
3.2.3 Diseño de la base de datos	39
3.2.4 Diseño del Contrato API y Enfoque <i>API-First</i>	42
3.2.5 Diseño de la interfaz	44
3.2.6 Control de versiones	47
4 Tecnologías usadas	49

4.1	Servidor back-end	49
4.2	Base de datos	50
4.3	Almacenamiento de objetos	52
4.4	Aplicación front-end	53
5	Implementación	55
5.1	Entorno de desarrollo	55
5.1.1	Herramientas y versiones	55
5.1.2	Ejecución local	55
5.1.3	Migraciones	56
5.1.4	Contrato API y generación de DTO	56
5.2	Servidor back-end	56
5.2.1	Estructura	56
5.2.2	Implementación de la seguridad	58
5.2.3	Implementación de la lógica de ligas	58
5.2.4	Generación de jornadas y partidos	59
5.2.5	Gestión de resultados y actas	59
5.2.6	Gestión de errores de validación	60
5.2.7	Pruebas y validación	60
5.3	Base de datos	60
5.4	Almacenamiento de objetos	61
5.5	Interfaz multiplataforma	62
5.5.1	Navegación	62
5.5.2	Comunicación con la API	62
5.5.3	Estilo de la interfaz	62
5.5.4	Flujos por rol	63
6	Resultado	65
6.1	Cambios aplicados	65
6.2	Desviaciones respecto a la planificación inicial	65
6.2.1	Tecnología	65
6.3	Funcionalidades implementadas	66
6.4	Funcionalidades no implementadas	67
7	Conclusiones	69
7.1	Cumplimiento de objetivos	69
7.2	Resultados obtenidos	69
7.3	Mejoras aportadas con respecto a las alternativas	70
7.4	Limitaciones	70
7.5	Aprendizaje	70
7.6	Trabajo futuro	71
A	Formularios de opinión	73
B	Resultados de los formularios	75
B.1	Respuestas a las preguntas abiertas	75
C	Interfaz Winner	77

ÍNDICE GENERAL

v

D Interfaz Clupik	81
E Requisitos	87
F Cronograma	89
G Diseño de la base de datos	91
H Contrato API	93
I Diseño del sistema	95
Bibliografía	101

ACRÒNIMS

TFG Treball Final de Grau

UIB Universitat de les Illes Balears

OUSIS Oficina d'Universitat Saludable i Sostenible

LVUIB Liga de Voleibol de la Universitat de les Illes Balears

SaaS Software as a Service, Software como Servicio

OS Object Storage, Almacenamiento de Objetos

BD Base de Datos

UX User Experience, Experiencia de Usuario

DTO Data Transfer Object, Objeto de Transferencia de Datos

ORM Object-Relational Mapping, Mapeo Relacional de Objetos

ABAC Attribute-Based Access Control, Control de Acceso Basado en Atributos

XSS Cross-Site Scripting

FVIB Federació de Voleibol de les Illes Balears

FIVB Fédération Internationale de Volleyball

RFEVB Real Federación Española de Voleibol

ACID Atomicidad, Consistencia, Aislamiento y Durabilidad

JWT JSON Web Token

RESUMEN

Este trabajo consiste en la comparación de las aplicaciones utilizadas por la Oficina d'Universitat Saludable i Sostenible (OUSIS) para la gestión de la Liga de Voleibol de la Universitat de les Illes Balears (LVUIB) con respecto a nuevas posibles soluciones. Se basa en el análisis de los pros y contras de cada aplicación utilizada y ofrece una solución nueva y mejorada cuyas funcionalidades se basan en las necesidades y opiniones recogidas de los principales usuarios. También explica la gestión del proyecto que acarrea el desarrollo de la herramienta de software necesaria de forma completa: desde la recogida de requisitos a partir de encuestas de opinión, pasando por el diseño de la arquitectura e interfaz y acabando con la implementación y evaluación del comportamiento de la nueva solución.

RESUM

Aquest treball consisteix en la comparació de les aplicacions utilitzades per la OUSIS per a la gestió de la LVUIB pel que fa a noves possibles solucions. Es basa en l'anàlisi dels pros i contres de cada aplicació utilitzada i ofereix una solució nova i millorada les funcionalitats de la qual es basen en les necessitats i opinions recollides dels principals usuaris. També explica la gestió del projecte que comporta el desenvolupament de l'eina de programari necessària de manera completa: des de la recollida de requisits a partir d'enquestes d'opinió, passant pel disseny de l'arquitectura i la interfície i acabant amb la implementació i l'avaluació del comportament de la nova solució.

INTRODUCCIÓN

La gestión de ligas deportivas universitarias puede resultar complicada si no se anticipan varios puntos como lo pueden ser: qué deportes ofrece la universidad interesada para poder celebrar ligas, qué formato seguirán dichas ligas (todos contra todos con grupos, fase eliminatoria, etc.), qué tipo de personas pueden participar (¿cualquier persona o solo gente relacionada con la universidad?), qué restricciones deben tener los equipos y jugadores que participan dentro de la liga, qué sistema de puntuación se debe utilizar, y muchos otros aspectos. Una planificación incorrecta en cualquiera de los puntos mencionados puede acarrear problemas de organización que lleven a un desarrollo de la liga que no acabe siendo satisfactorio para ningún interesado ya sea para los integrantes universitarios que participen o la misma universidad gestora.

Esto mismo es lo que ha ocurrido con la LVUIB. A lo largo de los años que se ha celebrado, los participantes y gestores se han dado cuenta de lo costoso que es encontrar un sistema a nivel de aplicación que permita organizar y gestionar una liga deportiva. Suelen ser necesarias muchas funcionalidades avanzadas y personalizadas que llevan a optar por soluciones que acaban necesitando una inversión monetaria, pero que aun así no acaban de satisfacer las necesidades de los participantes y usuarios de dicho sistema.

Debido a esta situación, se llegó a la conclusión de que era necesario desarrollar Sports League: un sistema de gestión centralizada que evite los problemas comentados y que, además, permita utilizar la aplicación como el único punto necesario de gestión.

Cabe remarcar que la OUSIS es el organismo encargado de fomentar y gestionar los eventos deportivos y relacionados con la salud que se celebran dentro del entorno de la Universitat de les Illes Balears (UIB), como pueden ser las ligas universitarias de fútbol, voleibol y pádel. Esta gestión no estuvo tan presente durante el primer año que se celebró la LVUIB, ya que fue un evento fomentado y gestionado por una asociación de alumnos creada al uso para dicho evento. En años posteriores, a medida que la LVUIB fue creciendo en popularidad y participantes, la OUSIS decidió involucrarse de lleno.

1.1 Alternativas actuales

Aunque actualmente podemos encontrar varias alternativas que aportan las funcionalidades básicas y genéricas necesarias para una correcta gestión de ligas deportivas, nos centraremos en las aplicaciones utilizadas en las temporadas anteriores de la LVUIB. Dichas aplicaciones son: Winner [1] y Clupik [2].

Las explicaciones de las funcionalidades de cada aplicación se basan en las imágenes que se pueden encontrar en los anexos correspondientes: C y D

1.1.1 Winner

Winner se utilizó en los primeros dos años de celebración de la LVUIB. De base, su uso es gratuito ofreciendo funcionalidades básicas, pero suficientes para organizar ligas deportivas amateurs. Las funcionalidades que nos ofrece son las siguientes:

Registro e inicio de sesión

El registro de sesión es sencillo. Permite crear una cuenta e iniciar sesión de forma básica utilizando un email y contraseña. También es posible iniciar sesión a través de una cuenta de Google.

Creación y gestión de ligas

Permite la creación de ligas ofreciendo una gran variedad de deportes y distintos tipos de participantes (individual, equipos, dobles, etc.). Es posible crear ligas dadas plantillas predefinidas o como consecución de fases de distintos formatos: árbol de eliminatoria y liga con o sin grupos. También es posible personalizar el sistema de puntuación y generar los partidos a partir de un calendario en el que se deciden las veces que se enfrentan cada equipo.

Contras: es una aplicación de gestión completamente local, es decir, la gestión recae sobre una sola persona que es la que posee el dispositivo donde se ha creado dicha liga. Para que otras personas puedan hacer un seguimiento de la liga es necesario compartir con ellos, de forma directa, el enlace que se genera al crear la liga y, a su vez, los interesados en visualizar la liga necesitan iniciar sesión. El sistema de puntuación es útil, pero básico; por ejemplo, en voleibol, no se podría otorgar una puntuación a cada equipo basándose en la diferencia de sets al acabar el partido, que es la forma oficial de puntuar un partido de voleibol.

Gestión de equipos y jugadores

Ofrece la posibilidad de registrar equipos y jugadores con sus respectivos nombres. También es posible asignarle logotipos a cada equipo y jugador a partir de imágenes básicas con colores genéricos y banderas de países que ofrece la aplicación

Contras: el registro de los equipos y jugadores es local, se hace todo en el dispositivo del administrador y genérico, tan solo permite registrar el nombre completo del jugador o equipo y seleccionar una imagen de un catálogo predeterminado; lo que puede ocasionar confusión si se están gestionando varias ligas al mismo tiempo. La

personalización es limitada dado que no es posible asignar imágenes completamente personalizadas a los escudos y fotos de perfil.

Visualización de Resultados y Clasificación

Es posible registrar los resultados de los partidos de forma manual: ubicación, ganador y el minutaje de anotación de cada punto. Lo cual hace que se actualice la tabla de clasificación del grupo pertinente. En cada tabla clasificatoria es posible visualizar las estadísticas generales de los equipos, además, Winner permite ver la evolución de las estadísticas de los equipos durante el desarrollo de la liga mediante diferentes gráficos.

Contras: ninguno, es posible aprender de los puntos fuertes de esta sección para poder aplicarlos a un nuevo sistema.

Opcionalidad de pago

Adicionalmente, contamos con la opción de adquirir una versión premium de la aplicación en caso de querer ampliar las funcionalidades. Las ventajas que proporciona la aplicación con su versión de pago y las ofertas existentes las podemos ver en la figura C.5.

Contras: la versión de pago se basa en un servicio de suscripción que, aunque sea asequible, hace necesario un pago recurrente para mantener las funcionalidades extra.

Resumen general

Winner, es una aplicación de gestión que cubre con creces las necesidades de ligas de nivel amateur, pero no es suficiente para ligas deportivas que quieran profesionalizarse. La responsabilidad recae únicamente en la persona que posee la aplicación y dispositivo sobre el que se va a efectuar la creación, registro y gestión de la liga, no es posible compartir la carga de trabajo entre varios administradores. Esto hace que no sea posible escalar su uso en la LVUIB debido a la necesidad de cumplir con los estándares derivados de ligas profesionales, como las gestionadas por la Federació de Voleibol de les Illes Balears (FVIB)

1.1.2 Clupik

Clupik se presenta como un Software as a Service, Software como Servicio (SaaS) que ofrece la posibilidad de gestionar tanto clubes como ligas deportivas, pero en este caso, es necesario centrarse en la funcionalidad que más importa para este proyecto, la gestión de ligas. La principal función de esta herramienta es la de permitir a cualquier persona crear y administrar una liga deportiva. Para poder hacerlo, pone a disposición de los usuarios las siguientes funcionalidades:

Registro e inicio de sesión

Es muy similar, por no decir casi idéntico, al sistema de Winner: creación de cuenta e inicio de sesión a partir de un email y una contraseña. Aunque en este caso no tenemos la posibilidad de enlazar una cuenta de Google. La parte positiva es que el inicio de sesión no es un requisito para poder consultar ligas públicas.

Creación y gestión de ligas

Se puede ver que la creación de una liga es similar a Winner aunque con algunas diferencias. Al crear la liga puedes elegir entre deportes y juegos de competición, cosa que Winner no contempla; aunque Clupik no permite elegir el formato de participación, solo se puede jugar dentro de un equipo, es decir, no se puede participar en una liga como un participante único o como una pareja tal y como permite hacerlo Winner. Clupik, al igual que Winner, permite la creación de ligas como sucesiones de fases, pero Clupik no tiene plantillas predefinidas como lo hace Winner. Es posible definir los partidos en los dos tipos de fases como podemos observar en las figuras D.2d y D.2e. Por último, es posible editar manualmente los equipos que se enfrentan en cualquier partido generado.

Contras: la mayor contraprestación que se puede apreciar es que para gestionar una liga, desde el papel de administrador, es necesaria una aplicación adicional, que se sincroniza con la aplicación principal, a la que existe para poder visualizar la liga desde el papel de espectador o participante. Y también existe otra aplicación para los árbitros que se centra únicamente en la gestión de los partidos. En resumen: existen 2 aplicaciones diferentes para asegurar una correcta gestión de la liga tanto desde el punto de vista de un administrador como del de un árbitro

Gestión de equipos y jugadores

La creación tanto de equipos como jugadores es muy básica: nombre en caso de los equipos y nombre y apellido en caso de jugadores.

Contras: no es posible personalizar una foto de perfil de jugador o escudo del equipo con las funcionalidades gratuitas de Clupik.

Visualización de Resultados y Clasificación

Para poder ver una liga, primero es necesario buscarla por su nombre o el nombre del organizador. Una vez que seleccionas una liga puedes consultar: los próximos partidos y los resultados de los partidos ya jugados, los diferentes grupos que hay, los equipos existentes, los jugadores por equipo, la tabla clasificatoria para cada grupo y las estadísticas de la liga. También es posible visualizar las jornadas que constituyen la liga y qué partidos se celebran en cada jornada.

Cabe destacar que existe una aplicación adicional para poder visualizar el desarrollo de la liga, aunque es su única funcionalidad. La visualización incluye todas las ligas activas de la UIB, aunque es posible aplicar una serie de filtros para centrarnos únicamente en el contenido que nos interesa (Ver D.6). También tiene la opción de iniciar sesión con unas credenciales válidas de la UIB para ver el contenido relacionado con la participación como usuario, por ejemplo: el equipo donde juega el usuario o los partidos en los que participa el equipo del usuario.

Contras: el principal punto negativo de esta sección es la existencia de dos aplicaciones diferentes. Esto puede hacerle creer al usuario que la segunda aplicación es una versión incompleta de la primera o que incluso crea que la primera es algo completamente agnóstico a la UIB al carecer el diseño propio de esta. Además, acarrea una carga cognitiva a los usuarios porque, al existir dos opciones no es intuitivo saber cuál de las dos ofrece toda la información de forma correcta.

Opcionalidad de pago

Se puede ver como Clupik ofrece dos tipos de planes de pago para extender las funcionalidades del sistema en caso de que las funcionalidades que vienen por defecto no sean suficientes para cumplir con las necesidades de gestión. Observamos como la oferta del plan *TOP* ofrece un conjunto de funcionalidades que permiten una completa gestión de ligas a todos los niveles: árbitros, administradores, cuerpo técnico y demás opciones que ayudan a hacer una gestión profesional. Todas ofertas disponibles y las funcionalidades que ofrecen se pueden ver en la Figura D.5.

Contras: el único contra y el más claro de esta sección es el elevado coste que ha supuesto el uso de este sistema durante la última temporada.

Uso en la LVUIB

Winner fue la aplicación que se utilizó durante los dos primeros años de celebración de la LVUIB. El primer año que se celebró los organizadores decidieron utilizar las funcionalidades básicas, mientras que el segundo año se optó por la versión de pago debido a la necesidad de mayor personalización, mejora de la organización y eliminación de los frecuentes anuncios que resultaban una molestia para el principal administrador. La opción de compra que se adquirió fue una que ya no está disponible: una compra única con un precio no mayor a 15 €.

Clupik se ha utilizado durante el tercer año (coincidiendo con el año académico 2025-2026) y ha representado un cambio importante respecto a la primera alternativa. Se presenta como un ‘cambio importante’ porque ha sido la propia OUSIS la que ha decidido implicarse más en la organización y ha sido la principal gestora de la LVUIB esta última temporada. Además, el uso de Clupik provocado que existan un total de dos aplicaciones que sirven para consultar los datos de la liga. Aunque la segunda solo muestra datos relacionados con los próximos partidos y la tabla de clasificación. Es decir, la segunda aplicación es una versión reducida de la primera añadiendo un sistema completo de filtros además de mostrar otro contenido referente a la OUSIS y presentar un diseño similar al de aplicaciones de la UIB.

La OUSIS optó por el plan de pago más completo y funcional: el plan *TOP*. Cabe destacar que el plan obliga a hacer el pago de forma anual. Esto hace que, como la LVUIB se celebra durante los dos semestres de un año académico, el coste total del sistema ha sido de 599.04 €.

Para acabar de entender esta sección la Tabla 1.1 compara las aplicaciones, sus funcionalidades y cuáles de estas funcionalidades se pretenden mejorar con el nuevo sistema.

Aplicaciones Funcionalidades	Winner	Clupik
Registro e inicio de sesión	No	No
Creación y gestión de ligas	Parcial	Parcial
Gestión de equipos y jugadores	Sí	Sí
Visualización	Parcial	Sí
Opcionalidad de pago	Sí	Sí

Cuadro 1.1: Comparativa de las alternativas

1.2 Motivación y objetivos

La motivación principal de este Treball Final de Grau (TFG) es la de diseñar e implementar un sistema gestor de ligas deportivas que aporte una mejora con respecto a las alternativas que se han estado utilizando hasta ahora dentro de la LVUIB. Esta mejora se basará en las opiniones de los usuarios que utilizaron una de las alternativas.

Los objetivos principales que han buscado cumplir a través del desarrollo de Sports League es:

Objetivo General

Desarrollar un sistema de gestión de ligas deportivas universitarias mejorando la comunicación y los flujos de trabajo entre todos los interesados. Y, además, asegurar que el sistema no requiere de herramientas externas ni diversas aplicaciones para poder funcionar.

Objetivos Específicos

OE1 Recopilar las necesidades de los distintos perfiles dentro de la LVUIB.

OE2 Diseñar un sistema que permita cumplir con las necesidades recopiladas.

OE3 Implementar y desarrollar este sistema definido.

OE4 Validar el correcto funcionamiento del sistema.

OE5 Crear una estrategia de despliegue escalable y robusta.

OE6 Documentar el sistema y facilitar el entendimiento a los interesados.

1.3 Estructura del documento

Este documento está estructurado siguiendo el desarrollo del proyecto, desde la planificación hasta las conclusiones finales.

En el capítulo 2 se detalla la planificación del proyecto seguida para completarlo, que incluye: interesados, el ciclo de vida, entregables, estimación de tiempo, gestión de calidad y gestión de cambios.

En el capítulo 3 se describen las especificaciones y el diseño del sistema tanto a nivel funcional como arquitectónico. Explica el proceso de recogida de requisitos a partir del uso de formularios de satisfacción y el diseño del sistema necesario para cumplir con dichos requisitos.

En el capítulo 5 se muestra cómo ha sido la implementación del sistema: arquitectura de software, patrones de diseño y principios SOLID [3] que se han aplicado.

En el capítulo 6 se muestran los resultados obtenidos, las desviaciones respecto a la planificación inicial y el alcance final del proyecto.

Finalmente, en el capítulo 7 se exponen las conclusiones derivadas del trabajo, el aprendizaje adquirido y las líneas futuras de mejora y continuación.

PLANIFICACIÓN DEL PROYECTO

2.1 Metodología

Para el desarrollo del proyecto se ha usado una metodología de desarrollo en cascada. Este enfoque resulta especialmente adecuado para un proyecto desarrollado por una única persona, en el que no existe un equipo multidisciplinar ni un cliente con requerimientos de actualización continua, sino un conjunto de objetivos definidos de antemano que deben alcanzarse de manera progresiva y controlada.

La estructura en fases facilita la planificación, la organización del trabajo y la evaluación del progreso, permitiendo detectar desviaciones en etapas tempranas y reducir así el riesgo de errores acumulados. Asimismo, la separación entre análisis, diseño, implementación, validación y documentación favorece una gestión más rigurosa del tiempo y de los recursos, y permite alinear cada hito del proyecto con los objetivos específicos establecidos al inicio de la investigación (Ver 1.2).

Las principales fases con las que cuenta el proyecto son las siguientes:

1. **Análisis:** definir el alcance del proyecto y las funcionalidades y capacidades con las que debe contar el sistema.
2. **Diseño del sistema:** esbozar la estructura y arquitectura del sistema y sus distintos componentes.
3. **Implementación:** aplicar el análisis y diseño completado para desarrollar el sistema y convertirlo en una aplicación funcional.
4. **Validación:** evaluar el sistema implementado para asegurar su correcto funcionamiento.
5. **Documentación:** registrar el trabajo completado en documentación formal y entregar el sistema desarrollado.

2.2 Entregables

Además de la planificación y el seguimiento del proyecto, se han definido una serie de entregables que permiten verificar el progreso y la evolución del sistema a lo largo de su desarrollo. Estos entregables incluyen la especificación de requisitos, el diseño de la arquitectura, el contrato de la API, la estructura de la base de datos, el código fuente de la aplicación, la configuración del entorno de despliegue y la documentación final del trabajo. Cada uno de estos artefactos responde a una fase concreta del proyecto y constituye un resultado intermedio que valida el avance del desarrollo antes de pasar a la siguiente etapa.

En la lista siguiente se pueden encontrar todos los entregables necesarios para este proyecto:

1. Requisitos funcionales.
2. Requisitos no funcionales.
3. Diagrama de casos de uso.
4. Diagrama usuario-sistema para un caso de uso crítico.
5. Diagrama C4 con los primeros tres niveles.
6. Modelo de datos.
7. Código fuente.
8. Contrato API.
9. Configuración de desarrollo local y despliegue.
10. Migraciones de la base de datos.
11. Documentación final
12. Resultados de validación del sistema.

2.3 Planificación temporal

En la Figura F.1 se muestra el cronograma del proyecto, donde se representan las diferentes fases del desarrollo y la secuencia temporal en la que se han llevado a cabo. Este esquema permite visualizar de forma clara el orden de ejecución de cada etapa, así como los hitos principales que marcan el avance del trabajo y la transición entre una fase y la siguiente. La planificación se ha estructurado de manera que responda tanto a la evolución técnica del sistema como al calendario académico del TFG, garantizando que cada bloque de trabajo se desarrolle en un periodo realista y con una carga de esfuerzo adecuada. En este sentido, la duración de las fases de análisis y desarrollo ha tenido en cuenta la necesidad de dedicar tiempo al estudio para los exámenes y para el aprendizaje de tecnologías, lo que ha condicionado la progresión del proyecto y ha motivado una distribución temporal más equilibrada entre trabajo académico y desarrollo del sistema.

2.4 Hitos principales

Tal y como se muestra en la Figura F.1, los hitos marcan la finalización formal de cada fase del proyecto. Cada hito se considera alcanzado cuando sus entregables asociados están terminados y verificados:

H1: Requisitos del sistema definidos

Se alcanza al completar y consolidar el catálogo de requisitos funcionales y no funcionales, junto con el diagrama de casos de uso y el diagrama usuario-sistema para el caso crítico. Se da por concluido cuando el alcance queda delimitado sin ambigüedades técnicas.

H2: Sistema diseñado

Se cumple al finalizar el modelo de datos, el contrato de la API y los tres primeros niveles del modelo C4. El criterio de cierre exige que la arquitectura soporte íntegramente los requisitos definidos en el hito anterior.

H3: Sistema implementado

Se alcanza cuando el código fuente de los módulos y las migraciones de la base de datos están completos, integrados en el repositorio y compilan sin errores, implementando la lógica acordada en el diseño y el contrato API.

H4: Sistema validado

Se marca como completado tras la ejecución de las pruebas y la obtención del documento de resultados de validación, confirmando que las funcionalidades operan según lo previsto y que no existen fallos críticos bloqueantes.

H5: Entorno y despliegue configurados

Se cumple al verificar la configuración de desarrollo local y generar los artefactos o contenedores de despliegue, comprobando que el sistema validado se ejecuta y responde correctamente en un entorno controlado.

H6: Proyecto documentado

Se alcanza con la redacción, revisión y maquetación definitiva de la memoria del trabajo, conteniendo todos los diagramas, especificaciones y resultados técnicos listos para su entrega académica.

2.5 Gestión de riesgos

Los riesgos identificados durante la planificación del proyecto se han clasificado en función de su probabilidad de aparición y del impacto potencial sobre el alcance, la calidad y la ejecución temporal del trabajo. Los cambios en los requisitos se consideran

2. PLANIFICACIÓN DEL PROYECTO

un riesgo de baja prioridad, ya que el análisis inicial, basado en formularios y perfiles de usuario, ha permitido definir de forma estable los objetivos funcionales del sistema. En cambio, los riesgos tecnológicos presentan una prioridad alta, debido a la dependencia de librerías, frameworks y herramientas externas que pueden sufrir cambios de versión, introducir vulnerabilidades o requerir ajustes de compatibilidad. Asimismo, la carga académica del estudiante y la necesidad de compatibilizar el desarrollo con los exámenes representan un riesgo de cronograma de prioridad media-alta, ya que pueden afectar a la secuencia de trabajo si no se gestionan con márgenes temporales adecuados. Por último, los riesgos asociados a la integración de servicios, la seguridad de acceso y la deuda técnica requieren una supervisión continua, dado que su materialización podría afectar tanto a la estabilidad del sistema como a su mantenibilidad futura. Toda esta información se puede consultar en detalle

Tipo de riesgo	Descripción	Probabilidad	Impacto	Mitigación
Cambios en requisitos	Modificaciones significativas una vez definidos los formularios y objetivos del sistema.	Baja	Medio	Se establece un alcance inicial definido y requisitos priorizados para evitar ampliaciones no controladas.
Dependencias tecnológicas	Cambios de versión, vulnerabilidades o incompatibilidades de librerías y frameworks.	Media	Alto	Uso de versiones concretas, revisión de dependencias y validación tras cambios.
Carga académica	Retrasos por exámenes y otras asignaturas del curso.	Alta	Medio-Alto	Planificación por fases con margen temporal y distribución equilibrada del trabajo.
Complejidad técnica	Dificultad derivada de integrar varios módulos funcionales en el sistema.	Media	Alto	Diseño previo, separación por dominios y desarrollo incremental.
Integración entre componentes	Problemas de comunicación entre front-end, back-end, base de datos y almacenamiento.	Media	Medio-Alto	Contrato API claro y pruebas de integración.
Seguridad y permisos	Accesos no autorizados o fallos en la gestión de roles y permisos.	Media	Alto	Validaciones de acceso y control por roles.
Deuda técnica	Soluciones rápidas que comprometen la mantenibilidad futura del sistema.	Media	Medio	Revisión continua de la arquitectura y evitación de soluciones temporales.
Desviación de alcance	Añadir funcionalidades no previstas y retrasar el proyecto.	Media	Medio	Definir requisitos esenciales y priorizar el alcance.

Cuadro 2.1: Riesgos identificados y estrategias de mitigación

ANÁLISIS Y DISEÑO DEL SISTEMA

En este capítulo veremos todos los pasos que se han tomado para completar la especificación y diseño del sistema. Abarca desde el análisis de stakeholders y recogida de requisitos a partir de formularios de opinión hasta el diseño completo del sistema.

3.1 Especificación

3.1.1 Análisis de los interesados

Entre los interesados del proyecto: todas aquellas partes que se ven, de una forma u otra, afectadas por el ciclo de vida del proyecto, podemos apreciar dos grupos de interesados claramente divididos. Por un lado, tenemos los interesados institucionales y académicos, que velan por el correcto desarrollo del proyecto; mientras que, por otro lado, están los usuarios finales, aquellos que interactuarán directamente con el sistema a través de los distintos roles definidos. Todos los interesados quedan explicados en la tabla 3.1.

Cuadro 3.1: Stakeholders del sistema

Stakeholder	Usuario del sistema	Descripción del Rol	Intereses y Objetivos Principales
OUSIS	No	Oficina de la UIB beneficiada por el desarrollo del proyecto	Obtener una herramienta de bajo coste y funcional que digitalice la gestión deportiva de la universidad, garantizando la viabilidad técnica y la mejora de los procesos actuales.
Tutor académico / Tribunal	No	Oficina de la UIB beneficiada por el desarrollo del proyecto	Garantizar que el proyecto cumple con los requisitos metodológicos, de arquitectura e ingeniería de software exigidos para la obtención del título.
Usuario / Espectador	Sí	Usuario común del sistema con o sin cuenta registrada	Consultar y ver los datos públicos de las ligas existentes.
Jugador	Sí	Participante inscrito en una o varias ligas.	Consultar las fechas de sus próximos encuentros, revisar la clasificación de su equipo y ver normativas.

Continúa en la página siguiente

Cuadro 3.1 – continuación de la página anterior

Stakeholder	Usuario del sistema	Descripción del Rol	Intereses y Objetivos Principales
Capitán	Sí	Representante del equipo, encargado de la gestión interna y la comunicación con otros equipos.	Inscribir a sus jugadores fácilmente y acordar fechas o modificaciones de partidos con otros capitanes de forma eficiente.
Árbitro	Sí	Responsable de la validez de los encuentros, la firma de actas y el reporte de resultados.	Disponer de una interfaz rápida para rellenar actas digitales y registrar el resultado de los partidos sin fricción.
Administrador	Sí	Gestor principal encargado de crear competiciones, calendarios y mantener el orden.	Automatizar la generación de calendarios, centralizar la resolución de incidencias y reducir el trabajo manual organizativo.

3.1.2 Recogida de requisitos

Los requisitos funcionales han sido generados basándonos en las opiniones que tenían los participantes de la LVUIB acerca del sistema que se ha estado utilizando: Clupik. Las respuestas hacen referencia tanto a las aplicaciones de Clupik como a la aplicación adicional de visualización de la UIB. Para completar la recogida de dichos requisitos se optó por la distribución de formularios de opinión y sugerencias. Las preguntas formuladas (que se pueden encontrar en los anexos) se agrupan en 4 formularios distintos según el rol que tuviera el participante dentro de la liga: jugador, capitán, árbitro y administrador.

Todos los formularios tienen en común estas cuestiones:

Aspectos cuantitativos

¿Cada cuánto usas la aplicación de la liga de la UIB o consultas la información de la liga a través de ella?

Pregunta de elección única cuyas opciones son:

- Más de 4 días a la semana.
- Menos de 4 días a la semana.
- Un día a la semana.
- Menos de 2 veces al mes.
- No sabía que había una aplicación o me informo a través de otras personas.

Con esta pregunta es posible hacerse una idea sobre lo importante que es el uso de la aplicación. Un uso bajo indica que la aplicación tiene una utilidad mínima o que incluso no la tiene, mientras que un uso alto indica que la aplicación es indispensable para la gestión de la liga.

¿A través de qué medio accedes principalmente a la aplicación?

Pregunta de elección única cuyas opciones son:

- Navegador web.
- Aplicación móvil.
- No accedo.

Nos aporta información con respecto al formato que usan los usuarios para acceder al sistema. Sirve para saber sobre qué dispositivos y plataformas enfocar el desarrollo del sistema.

Aspectos cualitativos

Usabilidad *Define del 1 al 10 lo cómodo que es utilizar la aplicación y navegar por ella.*
Pregunta con escala de Likert [4] (1-10) que va de *Pésima* a *Excelente*.

Accesibilidad *Define del 1 al 10 cuan fácil ha sido aprender a usarla.*

Pregunta con escala de Likert (1-10) que va de *He aprendido rápido* a *Todavía me cuesta saber cómo funciona*.

Manejo *Define del 1 al 5 cómo de rápido accedes al contenido/información que deseas ver.*

Pregunta con escala de Likert (1-5) que va de *Necesito menos de 3 clics* a *Me lío y tardo mucho*.

Diseño *¿Te gusta el diseño de la aplicación?*

Pregunta con escala de Likert (1-5) que va de *No es agradable de ver* a *Tiene un diseño atractivo*.

Estas cuatro preguntas aportan información en cuanto a la opinión que tiene los usuarios respecto a distintos aspectos funcionales de la aplicación.

Nota final y necesidad de un nuevo sistema

Nota final *¿Cómo puntuarías a nivel general la aplicación?*

Pregunta con escala de Likert (1-10) que va de *No me gusta mucho* a *No cambiaría nada*.

Tal y como indica la pregunta, muestra de forma directa la opinión que tienen los usuarios a nivel general.

¿Te gustaría una nueva aplicación que incluya nuevas y mejores funcionalidades?

Pregunta de elección única cuyas opciones son:

- Sí.
- No.

Esta sección permite saber la opinión general de los usuarios de la aplicación y si realmente necesitan un nuevo sistema que sustituya al actual.

Aspectos específicos

Cada formulario contiene una pequeña sección de cuestiones concretas relacionadas con el rol al cual se enfoca el formulario. Hay distintas preguntas según el rol.

Gestión equipos (Capitán) *¿Cómo puntuarías la gestión de tu equipo a través de la aplicación?*

Pregunta con escala de Likert (1-5) que va de *No la uso para nada* a *Excelente*.

Esta pregunta nos da información sobre el uso que hacen los capitanes de la aplicación.

¿Cómo puntuarías la gestión de los partidos a través de la aplicación? (Árbitro)

Pregunta con escala de Likert (1-5) que va de *Pésima* a *Excelente*.

Esta pregunta nos aporta información respecto a la facilidad que tienen los árbitros para arbitrar los partidos mediante la aplicación.

¿Usas algún dispositivo para rellenar el acta de los partidos? (Árbitro)

Pregunta de elección única cuyas opciones son:

- Tablet
- Móvil
- Solo uso acta en papel.

Con esta pregunta podemos saber si los árbitros utilizan algún dispositivo para rellenar el acta de los partidos o simplemente la rellenan a papel. Así podemos ver si es necesario incluir funcionalidades para la generación de actas digitales o incluso adaptar el diseño de la interfaz para otros tipos de dispositivos.

¿Cómo puntuarías la administración de la liga a través de la aplicación? (Administrador)

Pregunta con escala de Likert (1-5) que va de *Pésima* a *Excelente*.

Esta pregunta nos aporta información respecto a la facilidad que tiene los administradores para gestionar la liga a través de la aplicación.

¿Qué aspectos te gustaría que hubiera? Preguntas de elección múltiple cuyas opciones son:

Para jugadores:

- Aplicación móvil.
- Página web.
- Gestión del equipo en la propia aplicación.
- Solicitud para unirse a equipos dentro de una liga.
- Ninguna de las anteriores.

Para capitanes:

- Aplicación móvil.
- Página web.
- Gestión del equipo en la propia aplicación.
- Enviar solicitud para crear un equipo dentro de una liga.
- Enviar solicitud a jugadores para que se unan a mi equipo.
- Acuerdo de fecha del partido mediante propuestas.
- Crear incidencias: comportamiento árbitros, no imparcialidad, errores en el resultado, etc.
- Ninguna de las anteriores.

Para árbitros:

- Aplicación móvil.
- Página web.
- Crear incidencias: comportamiento de los jugadores de un equipo o del equipo en general.
- Registrar todos los datos del partido: puntos, sets, cambios, tarjetas, incidencias, etc. a través de la aplicación.
- Generación de acta electrónica al acabar un partido.
- Registrar mi disponibilidad para arbitrar.
- Enviar solicitud para arbitrar un partido.
- Rechazar el arbitraje concedido de un partido.
- Guardar mi firma para poder firmar los partidos de una manera más fácil y rápida.
- Ninguna de las anteriores.

Para administradores

- Aplicación móvil.
- Página web.
- Creación de ligas personalizadas: separación por distintas fases. Por ejemplo: liga 1, liga 2 (oro y plata), final four.
- Crear ligas a partir de formatos preestablecidos: Formato Real Federación Española de Voleibol (RFEVB), Formato Fédération Internationale de Volleyball (FIVB), etc.
- Personalización completa sobre las ligas: número de sets por partido, puntos por set, tiempos muertos.
- Resolución de incidencias creadas sobre jugadores o equipos con su correspondiente penalización.
- Asignar un árbitro a un partido de forma directa.
- Asignaciones automáticas de árbitros a partidos.
- Aceptar solicitudes de creación de equipos por parte de los jugadores.
- Ninguna de las anteriores.

Estas preguntas nos permiten saber qué funcionalidades les gustaría que hubiera a los usuarios según su rol. De esta forma es posible enfocar el diseño del sistema para cumplir con dicha separación de roles.

Preguntas abiertas

Otros aspectos *Pon cada funcionalidad que te gustaría que hubiera y que no haya aparecido en la lista anterior separada por comas. Ej.: Quiero [insertar funcionalidad] para poder/porque [insertar razón/justificación]*

Problemas encontrados *¿Qué aspectos has encontrado engorrosos o que no te parezcan bien? Por ejemplo: "Me cuesta llegar a la sección de clasificación de los equipos", etc.*

Preguntas de respuesta abierta que permiten a los encuestados detallar problemas específicos dentro de la aplicación además de especificar otras funcionalidades deseadas que no se contemplan en otras preguntas

En la siguiente tabla se muestra un resumen con las preguntas formuladas en los formularios:

3. ANÁLISIS Y DISEÑO DEL SISTEMA

Pregunta	Tipo	Rol
¿Cada cuánto usas la aplicación de la liga de la UIB o consultas la información de la liga a través de ella?	Elección única	Todos
¿A través de qué medio accedes principalmente a la aplicación?	Elección única	Todos
Usabilidad	Escala de Likert (1-10)	Todos
Accesibilidad	Escala de Likert (1-10)	Todos
Manejo	Escala de Likert (1-5)	Todos
Diseño	Escala de Likert (1-5)	Todos
Nota final	Escala de Likert (1-10)	Todos
¿Te gustaría una nueva aplicación que incluya nuevas y mejores funcionalidades?	Elección única	Todos
Gestión equipos	Escala de Likert (1-5)	Capitán
¿Cómo puntuarías la gestión de los partidos a través de la aplicación?	Escala de Likert (1-5)	Árbitro
¿Usas algún dispositivo para rellenar el acta de los partidos?	Elección única	Árbitro
¿Cómo puntuarías la administración de la liga a través de la aplicación?	Escala de Likert (1-5)	Administrador
¿Qué aspectos te gustaría que hubiera?	Elección múltiple	Todos
Otros aspectos	Respuesta abierta	Todos
Problemas encontrados	Respuesta abierta	Todos

Se puede apreciar que, para las preguntas de escala de Likert, la mayoría de las veces se usan escalas del al 10 a excepción de dos casos, que se usan escalas del 1 al 5: el diseño y el manejo de la aplicación. Esta diferencia está directamente relacionada con la naturaleza del aspecto evaluado. La mayoría de aspectos de escalas funcionan como evaluaciones estándar para facilitar a los encuestados la evaluación; pero, en los aspectos mencionados la escala funciona como una asociación directa con la pregunta. Para el manejo vemos como la escala representa la cantidad de clics necesarios para acceder a la información deseada donde una nota más baja indica una cantidad inferior de clics. Mientras que para el diseño se hace de este modo por comodidad del encuestado. En aspectos subjetivos como este, al encuestado le es más cómodo evaluar del 1 al 5 que del 1 al 10, ya que de esa forma acarrea una menor carga cognitiva al necesitar menos argumentos internos para evaluar el aspecto.

Los formularios fueron distribuidos a través de WhatsApp a los distintos grupos objetivos y se esperó a que hubiera una cantidad significativa de respuestas.

3.1.3 Análisis de las respuestas de los formularios

Las encuestas fueron respondidas por un total de 26 participantes: 16 jugadores, 2 árbitros, 6 capitanes y 2 administradores. Las respuestas presentadas en los formularios concluyeron en los siguientes resultados:

Aspectos cuantitativos

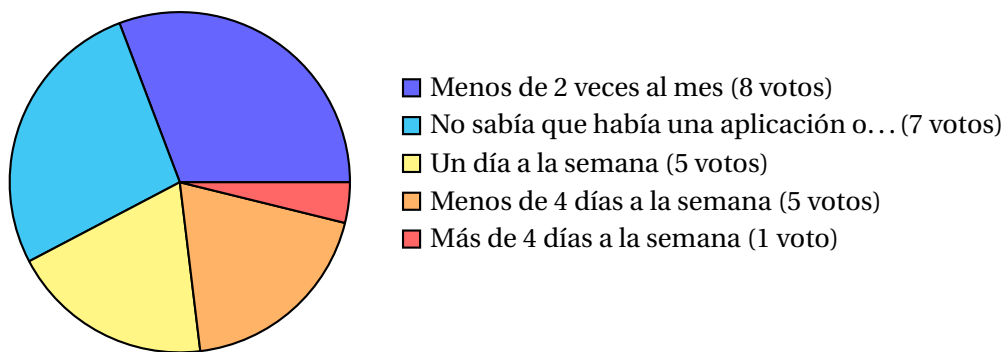


Figura 3.1: ¿Cada cuánto usas la aplicación de la liga de la UIB o consultas la información de la liga a través de ella?

La Figura 3.1 nos permite ver que la mayoría de los usuarios casi no usan la aplicación o que incluso no la usan para nada. Esto puede ser debido a la poca utilidad o facilidad de uso de la aplicación que encuentran en su uso cotidiano.

Esto nos puede indicar que la aplicación tan solo la usan los usuarios que estrictamente tienen que utilizarla: como los árbitros, para guardar los resultados de los partidos; o los administradores, para hacer un seguimiento continuo del correcto desarrollo de la liga.

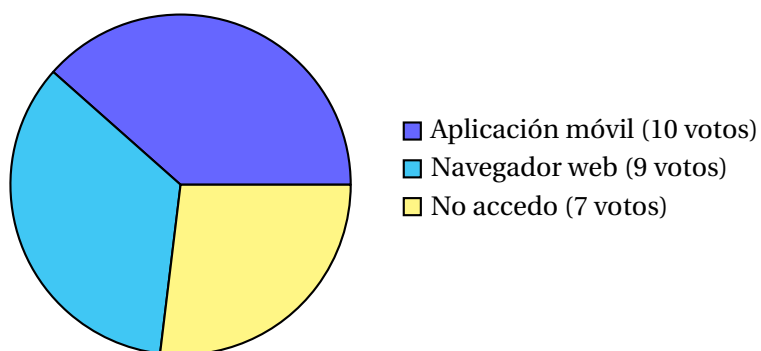


Figura 3.2: ¿A través de qué medio accedes principalmente a la aplicación?

La Figura 3.2 nos confirma el uso multiplataforma que hacen los usuarios de la aplicación, ya sea para ver ciertos datos de forma rápida con dispositivos móviles o consultas más precisas y extensas con el navegador web. Este dato nos informa sobre la necesidad de un sistema disponible en dispositivos varios.

Aspectos cualitativos

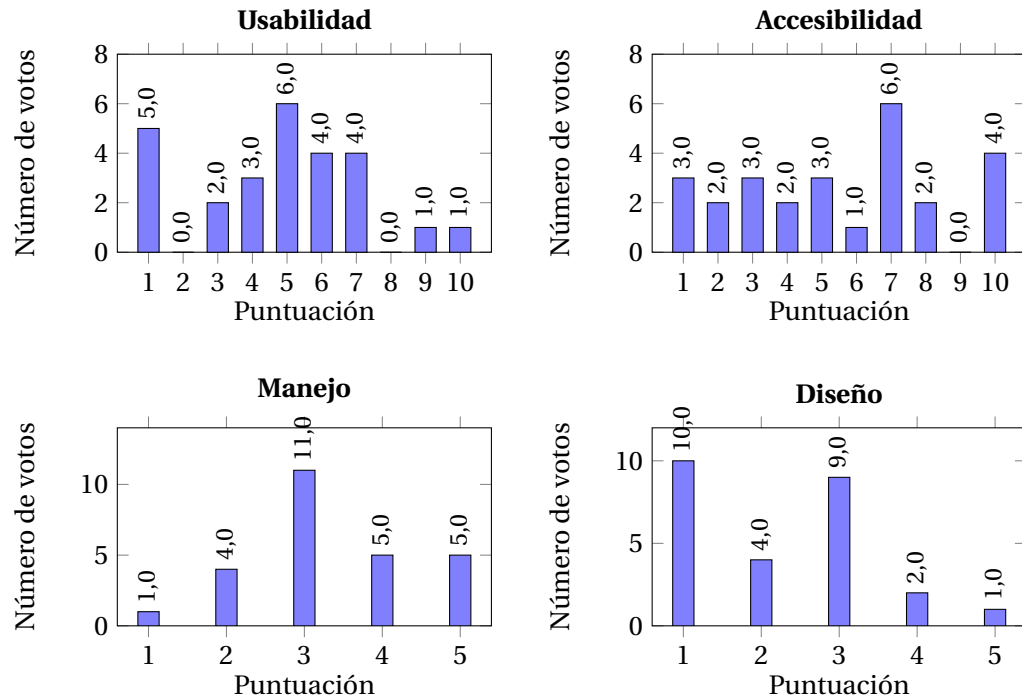


Figura 3.3: Aspectos comunes

Las cuatro Figuras de 3.3 nos detallan cómo los usuarios sienten la interactividad con la aplicación. Esto nos puede ayudar a saber qué debemos cambiar respecto a la aplicación actual y qué podemos replicar y mejorar.

Podemos apreciar que a la mayoría de los usuarios no les desagrada la usabilidad. Con esto podemos intentar seguir un esquema de uso similar con el que cuenta la aplicación.

La accesibilidad parece seguir un patrón uniforme que crece conforme nos acercamos a valores que reflejan una menor accesibilidad. Lo que nos dice que la accesibilidad es un punto a mejorar necesitando crear un sistema intuitivo que no le genere fricción al usuario.

El manejo es puntuado mayoritariamente con un valor medio, pero que decrece conforme nos acercamos a las mejores puntuaciones. Esto nos dice que necesitamos un sistema que ofrezca al usuario una distribución de los datos sencilla y entendible.

Por último, el diseño no es puntuado muy alto, lo que nos dice que nuestro sistema deberá tener una mejora visual que sea atractiva para los usuarios.

En la Figura 3.4 vemos cómo la administración y gestión deportiva a través de la aplicación no es ni lo mejor ni lo peor. No habría que aplicar mejoras respecto a lo existente, pero podemos añadir nuevas funcionalidades que faciliten el trabajo de los administradores.

Según la Figura 3.5, la gestión de los partidos parece ser excelente. Aunque cabe recalcar que los árbitros utilizan diversas aplicaciones para gestionar los partidos: tanto para arbitrar un partido como para anotar los resultados. Por ello existe la necesidad de hacer un sistema unificado para todos los interesados.

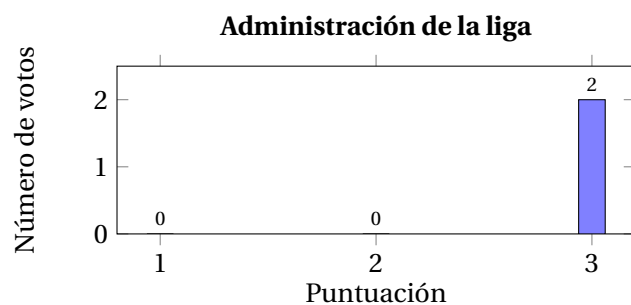


Figura 3.4: Valoración de la administración de la liga

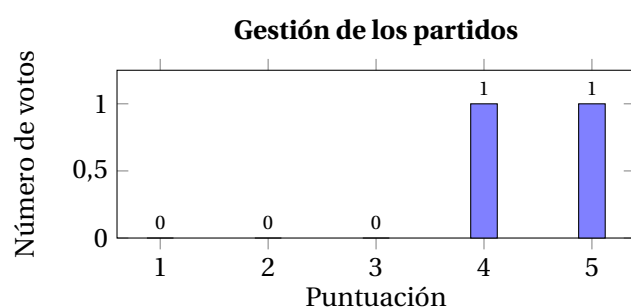


Figura 3.5: Valoración de la gestión de los partidos

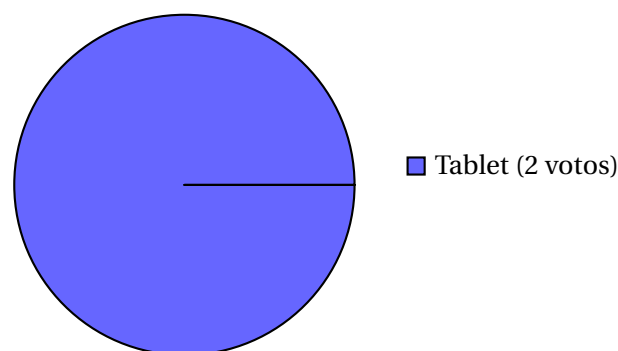


Figura 3.6: Dispositivo utilizado para rellenar el acta

La Figura 3.6 nos confirma la necesidad del sistema multiplataforma al observar el uso de tablets por parte de los árbitros para rellenar el acta de los partidos.

En la Figura 3.7 vemos como la opinión respecto a la gestión de equipos es muy baja. Esto nos lleva a deducir que puede ser interesante añadir funcionalidades para mejorar la interactividad y participación de los usuarios a la hora de gestionar los equipos deportivos.

Nota final y necesidad de un nuevo sistema

En las puntuaciones finales (Ver 3.8), observamos un patrón en el que destacan los valores bajos y donde los valores altos son insignificantes en comparación. Esto nos da el veredicto final y nos describe el descontento generalizado que hay con la aplicación.

La Figura 3.9 es la clave y el principal motivo del desarrollo de un nuevo sistema.

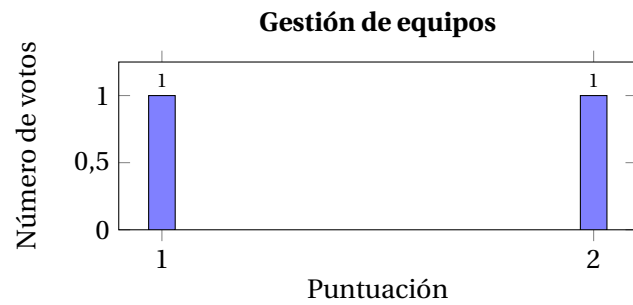


Figura 3.7: Valoración de la gestión de equipos

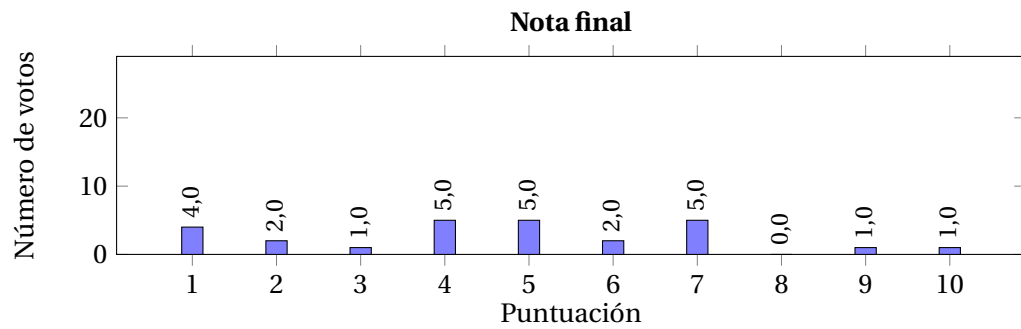


Figura 3.8: Nota final

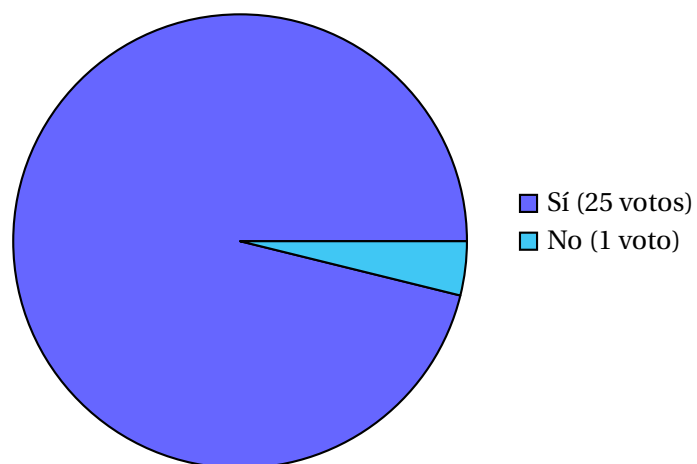


Figura 3.9: ¿Te gustaría una nueva aplicación que incluya nuevas y mejores funcionalidades?

Nos confirma que la gran mayoría de los usuarios buscan nuevas opciones que mejoren la experiencia que han tenido hasta ahora con el sistema actual.

Aspectos específicos

Ahora que sabemos que es necesario un nuevo sistema de gestión, con las siguientes cuestiones sabremos qué funcionalidades son necesarias para poder cumplir con las exigencias y carencias de los usuarios. Y, a partir de los resultados, seremos capaces de definir los requisitos funcionales del nuevo sistema.

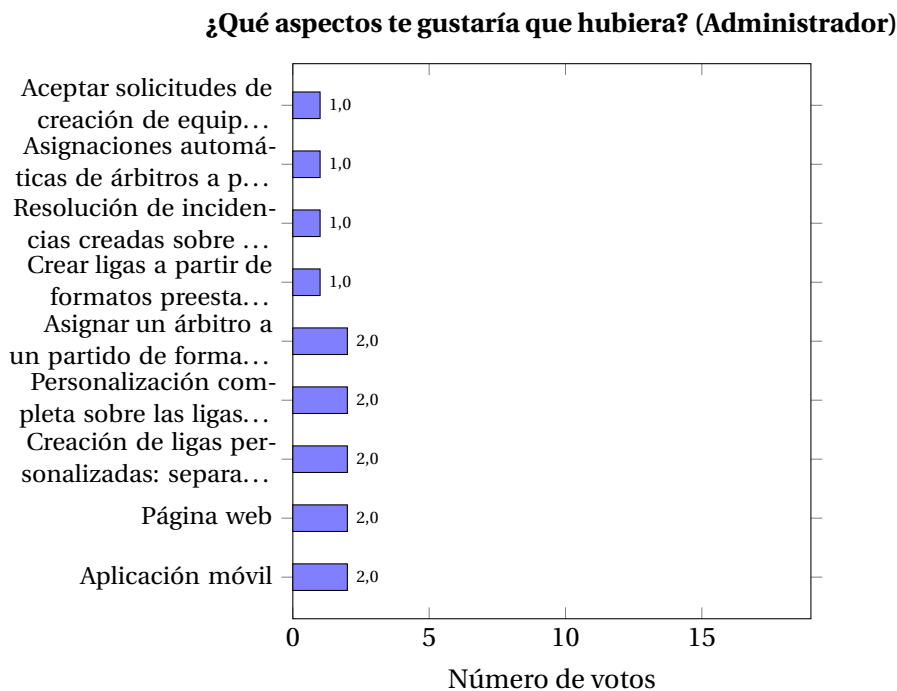


Figura 3.10: Nuevas funcionalidades: Administrador

Vemos como los administradores buscan una mayor versatilidad y facilidad a la hora de poder gestionar la liga desde cualquier dispositivo. Una gestión amena que no genere fricción con el usuario.

Mientras que los árbitros se centran principalmente en que puedan especificar su disponibilidad y que el sistema actúe acorde.

Por último, tanto los capitanes como los jugadores buscan una mayor interacción con el sistema a la hora de gestionar los equipos.

Preguntas abiertas

Finalmente, con la sección de respuestas abiertas los usuarios comentan problemas que no han podido destacar con las secciones anteriores.

El análisis de las respuestas abiertas revela tres áreas principales de mejora que justifican el desarrollo de la nueva plataforma.

En primer lugar, destaca la falta de adaptación al voleibol. Los usuarios señalan de forma recurrente que el sistema actual está enfocado al fútbol, lo que provoca errores y

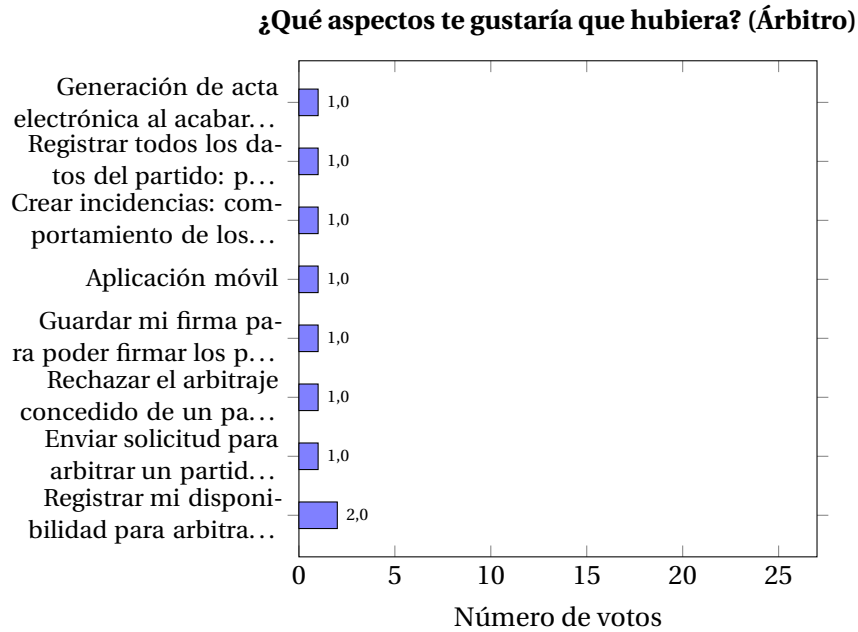


Figura 3.11: Nuevas funcionalidades: Árbitro

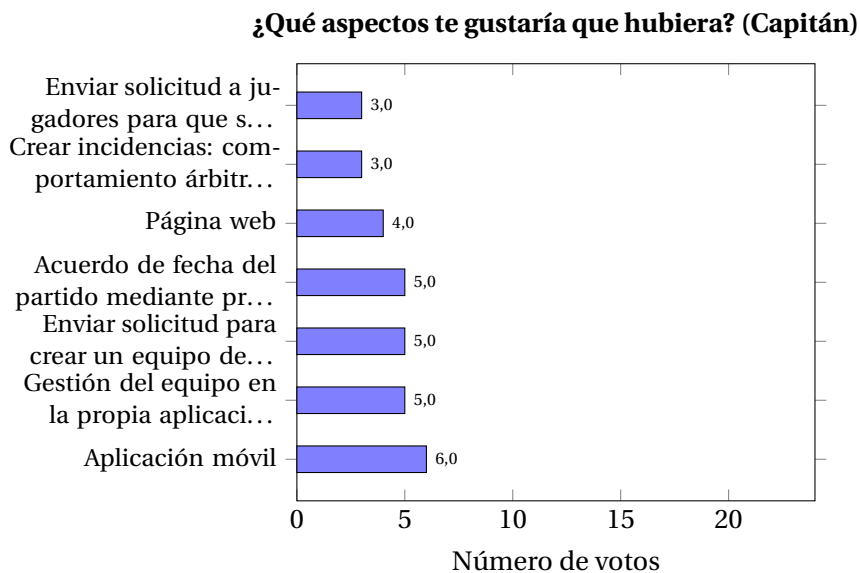


Figura 3.12: Nuevas funcionalidades: Capitán

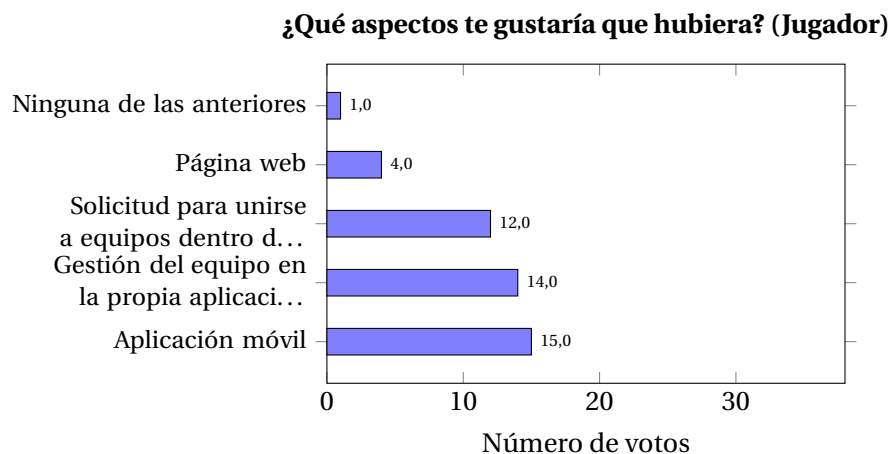


Figura 3.13: Nuevas funcionalidades: Jugador

confusión a la hora de registrar los resultados por sets, repartir los puntos y visualizar las clasificaciones.

En segundo lugar, se reportan notables deficiencias de rendimiento y usabilidad. Los encuestados mencionan lentitud general, fallos de carga en apartados como las notificaciones, errores en la actualización del estado de los partidos y dificultades técnicas para acceder a la aplicación en ciertos dispositivos móviles.

Por último, existe una clara demanda de mejores herramientas de gestión y comunicación. Las respuestas subrayan la necesidad de que los equipos puedan gestionar sus propias plantillas, recibir avisos de los próximos encuentros y contar con un sistema mucho más ágil para visualizar el calendario y asignar a los árbitros.

Para consultar el listado íntegro de las respuestas y los problemas específicos reportados por los usuarios, véase las listas que se encuentran en la página B.1.

Resumiendo, buscan un sistema multiplataforma, interactivo, abierto a participación y fácil de utilizar. Un sistema que refleje de forma mucho más clara el tipo de deporte que se está jugando y ofrezca funcionalidades que faciliten dicha gestión. Un sistema a prueba de fallos, eficiente, consistente y visualmente atractivo.

3.1.4 Reunión con la OUSIS

Al mismo tiempo que se empezó con la recogida de requisitos, se estableció contacto con la OUSIS. Habían escuchado que se estaba desarrollando este proyecto y decidieron contactar para aclarar ciertas dudas que tenían.

Tuvimos dos reuniones en las que no se hicieron ningún acta de reunión. En la primera, estuvo presente Raimon Massanet y consistió en una ronda de preguntas por parte de los responsables de la OUSIS: Joan Mercant Requena y Olalla Garcia Taibo. Fue necesario aclarar que las encuestas eran de carácter completamente anónimo y no se recogía ningún dato personal.

En la segunda reunión, Antonio Crespí presente, se estableció un contacto profesional para utilizar los resultados del proyecto como una base para implementar un nuevo sistema de gestión de actividades deportivas dentro de la UIB. Algunas de sus peticiones fueron: poder utilizar la aplicación para gestionar otras actividades que no

sean ligas deportivas y extender el uso de la aplicación a más deportes, ya que en un principio la aplicación estaba enfocada únicamente a voleibol. Debido a esto, se intentó que todo el proyecto fuera capaz de soportar estos requisitos a futuro, pero sin alterar el alcance base del sistema.

3.1.5 Requisitos funcionales

A partir de los resultados de las encuestas y las reuniones con la OUSIS se han sintetizado los siguientes requisitos funcionales y no funcionales:

Cuadro 3.2: Requisitos funcionales del sistema

Código	Entidad	Descripción
RF-USR-1	Usuario	El sistema debe permitir a los usuarios poder crear una cuenta para registrar sus datos en el sistema.
RF-USR-2	Usuario	El sistema debe permitir a los usuarios registrar la siguiente información: nombre completo, email, contraseña, sexo, referencias de las distintas licencias arbitrales que pueda tener un usuario y foto de perfil.
RF-USR-3	Usuario	El sistema debe permitir que los usuarios puedan indicar el deporte al que corresponden sus distintas licencias arbitrales.
RF-USR-4	Usuario	El sistema debe permitir a los usuarios iniciar sesión con una cuenta ya creada dadas sus credenciales: nombre de usuario o email y contraseña.
RF-USR-5	Usuario	El sistema debe permitir a los usuarios con sesión iniciada crear ligas deportivas. El usuario que crea una liga se convierte automáticamente en administrador de la liga.
RF-USR-6	Usuario	El sistema debe permitir a los usuarios sin cuenta poder ver ligas ya creadas.

Continúa en la página siguiente

Cuadro 3.2 – continuación de la página anterior

Código	Entidad	Descripción
RF-USR-7	Usuario	El sistema debe permitir que los usuarios puedan almacenar su firma como una imagen que pueda ser reutilizada para firmar en los partidos en caso de ser capitán o árbitro.
RF-LIG-1	Liga	El sistema debe permitir crear ligas con la siguiente información: nombre, descripción, icono, banner, ubicación, fecha de inicio, fecha de fin y fecha máxima para inscribirse.
RF-LIG-2	Liga	El sistema debe permitir configurar los siguientes parámetros dentro de una liga: categoría (Masculina, Femenina o Mixta), número de integrantes femeninos mínimos en caso de que la categoría sea mixta, número mínimo y máximo de integrantes en cada equipo y duración de las jornadas en semanas.
RF-LIG-3	Liga	El sistema debe permitir implementar un sistema de roles dentro de las ligas: Administrador, Árbitro, Capitán y Jugador.
RF-LIG-4	Liga	El sistema debe permitir a los usuarios con sesión iniciada unirse a ligas creadas. Al unirse se convierte en Jugador dentro de esa liga.
RF-LIG-5	Liga	El sistema debe permitir que los participantes de una liga puedan crear incidencias dentro de esta. Las incidencias deben ir acompañadas de una descripción.
RF-LIG-6	Liga	El sistema debe permitir que los administradores de una liga puedan expulsar a los participantes de esta.
RF-LIG-7	Liga	El sistema debe permitir que una liga esté conformada por distintas fases.

Continúa en la página siguiente

Cuadro 3.2 – continuación de la página anterior

Código	Entidad	Descripción
RF-LIG-8	Liga	El sistema debe permitir que un usuario pueda elegir ver únicamente las ligas en las que está participando.
RF-EQU-1	Equipo	El sistema debe permitir a los usuarios dentro de una liga crear y enviar solicitudes a los administradores para poder crear equipos dentro de la liga.
RF-EQU-2	Equipo	El sistema debe poder permitir crear equipos con la siguiente información: nombre, iniciales, descripción, lema, color primario, color secundario y escudo.
RF-EQU-3	Equipo	El sistema debe permitir al administrador de una liga aceptar o rechazar solicitudes de creación de equipo. En caso de ser rechazada se debe poder incluir un mensaje con la justificación pertinente.
RF-EQU-4	Equipo	El sistema debe permitir que, al aceptar una solicitud de creación de equipo, el solicitante se convierta automáticamente en el capitán del equipo solicitado.
RF-EQU-5	Equipo	El sistema debe permitir que los jugadores que no tengan equipo puedan enviar una solicitud al capitán de un equipo para poder unirse a este.
RF-EQU-6	Equipo	El sistema debe permitir que los capitanes puedan aceptar o rechazar las solicitudes de unión de su equipo.
RF-EQU-7	Equipo	El sistema debe permitir que los capitanes de los equipos puedan enviar solicitudes a jugadores sin equipo para que se unan a este.

Continúa en la página siguiente

Cuadro 3.2 – continuación de la página anterior

Código	Entidad	Descripción
RF-EQU-8	Equipo	El sistema debe permitir que los jugadores puedan aceptar o rechazar las solicitudes para unirse a un equipo.
RF-EQU-9	Equipo	El sistema debe permitir que los capitanes de un equipo puedan expulsar a los jugadores de este.
RF-EQU-10	Equipo	El sistema debe permitir que los jugadores puedan elegir un número de dorsal dentro de su equipo que no se haya elegido antes.
RF-EQU-11	Equipo	El sistema debe permitir que el capitán de un equipo pueda eliminar el equipo que lidera.
RF-ROL-1	Rol	El sistema debe permitir que los participantes de una liga puedan tener uno o más roles asignados.
RF-ROL-2	Rol	El sistema debe permitir que el creador de una liga pueda asignar el rol de Administrador a otros participantes de esta. De este modo se puede repartir el trabajo que conlleva gestionar la liga.
RF-ROL-3	Rol	El sistema debe permitir que los jugadores de una liga puedan crear solicitudes para que los administradores les puedan asignar el rol de Árbitro en dicha liga.
RF-ROL-4	Rol	El sistema debe permitir que los administradores rechacen o acepten las solicitudes de los jugadores para convertirse en árbitros. Dicha respuesta y solicitud cuenta con la misma información que las solicitudes para crear equipos.
Continúa en la página siguiente		

Cuadro 3.2 – continuación de la página anterior

Código	Entidad	Descripción
RF-ROL-5	Rol	El sistema debe permitir que los administradores puedan retirar el rol de Árbitro a los participantes árbitros que vea conveniente.
RF-ROL-6	Rol	El sistema debe permitir que los administradores puedan eliminar un equipo.
RF-FAS-1	Fase	El sistema debe permitir configurar cada fase con la siguiente información: nombre, fecha de inicio, fecha de fin, formato y su orden dentro de la secuencia de fases.
RF-FAS-2	Fase	El sistema debe permitir que cada fase pueda ser uno de los siguientes formatos: Torneo y Clasificatorio. En el formato Clasificatorio se debe especificar el número de ganadores que tendrá. Podrían añadirse nuevos formatos en un futuro.
RF-FAS-3	Fase	El sistema debe permitir que cada fase esté compuesta por una o más Rondas.
RF-RON-1	Ronda	El sistema debe permitir que para cada Ronda se pueda establecer el día en que empieza dicha Ronda.
RF-RON-2	Ronda	El sistema debe permitir que en cada ronda se celebren diversos partidos.
RF-RON-3	Ronda	El sistema debe permitir que los usuarios con rol de Administrador de una liga establezcan la disponibilidad de la ubicación de juego para cada Ronda.
RF-DIS-1	Disponibilidad	El sistema debe permitir que la disponibilidad de una ronda se establezca como un conjunto de franjas horarias.

Continúa en la página siguiente

Cuadro 3.2 – continuación de la página anterior

Código	Entidad	Descripción
RF-DIS-2	Disponibilidad	El sistema debe permitir que en cada franja horaria se pueda especificar el día, la hora y la duración de la franja.
RF-DIS-3	Disponibilidad	El sistema debe permitir que los administradores de una liga puedan establecer la disponibilidad de la ubicación de juego para cada ronda.
RF-DIS-4	Disponibilidad	El sistema debe permitir que cada usuario, en especial los capitanes y los árbitros, puedan establecer su disponibilidad para cada ronda respecto a la disponibilidad establecida por los administradores para la ubicación de juego.
RF-PAR-1	Partido	El sistema debe permitir que los partidos se generen de forma automática dependiendo del formato de la Fase.
RF-PAR-2	Partido	El sistema debe permitir que en cada partido se enfrenten un equipo local y un equipo visitante.
RF-PAR-3	Partido	El sistema debe permitir que las asignaciones de los equipos que se enfrentan en los partidos se pueda hacer de forma automática o manual.
RF-PAR-4	Partido	El sistema debe permitir que la fecha de un partido se establezca mediante un sistema de negociación entre los capitanes de los equipos que deben enfrentarse.
RF-PAR-5	Partido	El sistema debe permitir que los administradores de una liga puedan fijar la fecha de un partido que no se haya fijado.
Continúa en la página siguiente		

Cuadro 3.2 – continuación de la página anterior

Código	Entidad	Descripción
RF-PAR-6	Partido	El sistema debe permitir que a cada partido se pueda asignar un árbitro. El árbitro se puede asignar de forma manual por parte de los Administradores o de manera automática.
RF-PAR-7	Partido	El sistema debe permitir que la forma automática para asignar un árbitro a un partido se haga mediante un algoritmo que, para un árbitro con una cierta disponibilidad, se le asigne un partido cuya fecha coincida con alguna de las franjas disponibles del árbitro. El algoritmo también debe tener en cuenta la cantidad de partidos arbitrados por cada árbitro para poder repartir el arbitraje de manera equitativa.
RF-PRO-1	Propuesta	El sistema debe permitir que el proceso de negociación de la fecha de un partido consista en lo siguiente: el capitán del equipo local crea y envía una primera propuesta con la franja horaria disponible deseada al capitán del equipo visitante. El capitán del equipo visitante puede decidir aceptar o rechazar la propuesta, si la rechaza debe enviar una contrapropuesta. Este proceso se repite hasta que alguno de los dos acepte la propuesta.
RF-RES-1	Resultado	El sistema debe permitir que se puedan guardar un Resultado para cada partido.
RF-RES-2	Resultado	El sistema debe permitir que cada resultado contenga la siguiente información: puntuación del equipo local, puntuación del equipo visitante, observaciones encontradas durante el partido y firmas de los árbitros y capitanes antes y después del partido.
Continúa en la página siguiente		

Cuadro 3.2 – continuación de la página anterior

Código	Entidad	Descripción
RF-RES-3	Resultado	El sistema debe permitir que cada capitán y árbitro pueda firmar el acta de un partido desde su propio dispositivo.
RF-RES-4	Resultado	El sistema debe permitir que un partido se divida en distintos Periodos.
RF-RES-5	Resultado	El sistema debe permitir que cada periodo guarde la siguiente información: orden numérico del periodo dentro del partido, tipo de periodo según el deporte (set, mitad, cuarto, etc.) y puntuación de los dos equipos para ese periodo.
RF-PER-1	Periodo de Partido	El sistema debe permitir que cada Periodo contenga: la puntuación del equipo local y del equipo visitante para ese periodo, orden numérico del periodo dentro del partido y tipo de periodo según el deporte.
RF-PER-2	Periodo de Partido	El sistema debe permitir que cada periodo pueda tener Eventos.
RF-EVE-1	Evento de Partido	El sistema debe permitir que para cada evento se guarde la siguiente información: el tiempo en el que se genera respecto al inicio del partido, puntuaciones del equipo local y visitante para ese periodo y qué equipo ha sido artífice de dicho evento.

Continúa en la página siguiente

Cuadro 3.2 – continuación de la página anterior

Código	Entidad	Descripción
RF-EVE-2	Evento de Partido	El sistema debe permitir que cada evento pueda ser uno de tres tipos distintos: sustitución, sanción o tiempo muerto. Para la sustitución se debe especificar los dos jugadores que se intercambian. En la sanción se debe especificar una descripción y el tipo de sanción. Para los tiempos muertos se debe especificar la duración de dicho tiempo.

Podemos apreciar mejor los requisitos funcionales mediante el diagrama de casos de uso presente en el apéndice E.1

Cuadro 3.3: Requisitos no funcionales del sistema

Código	Tipo	Descripción
RNF-USA-1	Usabilidad	La aplicación móvil debe permitir a un árbitro registrar eventos y cerrar un acta en un máximo de 3 a 5 clics desde la pantalla principal.
RNF-USA-2	Usabilidad	La interfaz debe contar con un contraste adecuado para garantizar la legibilidad en entornos exteriores (pistas al aire libre con mucha luz).
RNF-SEG-1	Seguridad	El acceso a los endpoints críticos (como forzar un horario o modificar roles) debe estar protegido mediante autenticación por token (JWT) y validación estricta de roles y atributos: ABAC.
RNF-SEG-2	Seguridad	Las contraseñas de los usuarios no se almacenarán en texto plano bajo ninguna circunstancia; se utilizará un algoritmo de 'hashing' fuerte (como 'bcrypt').

Continúa en la página siguiente

Cuadro 3.3 – continuación de la página anterior

Código	Tipo	Descripción
RNF-SEG-3	Seguridad	Sanitización de entradas: Todos los datos enviados desde la aplicación móvil hacia la API deben ser validados y sanitizados para prevenir ataques de inyección SQL (gestionado a través del ORM de <i>Spring JPA</i>) y ataques XSS.
RNF-DEP-1	Fiabilidad	El sistema debe garantizar la atomicidad de las operaciones críticas (como el guardado del acta final). Si la conexión se pierde a mitad del envío, la base de datos debe revertir los cambios para evitar estados inconsistentes (actas a medias).
RNF-EFI-1	Eficiencia (Espacio)	Las imágenes subidas al sistema (como firmas de capitanes, fotos de perfil o logos de equipos) deben comprimirse para no superar un peso máximo de 200KB por imagen.
RNF-EFI-2	Eficiencia (Rendimiento)	Las consultas de alta frecuencia, como la visualización de la clasificación, deben resolverse en menos de 200 ms desde el back-end, apoyándose en la base de datos optimizada (vistas materializadas).
RNF-EFI-3	Eficiencia (Rendimiento)	La aplicación móvil debe arrancar y mostrar la pantalla de inicio (Dashboard) en menos de 3 segundos en condiciones de red estándar (4G/Wifi).

Continúa en la página siguiente

Cuadro 3.3 – continuación de la página anterior

Código	Tipo	Descripción
RNF-OPE-1	Operacional	Las constantes del sistema de competición (como puntos por victoria, duración de jornadas) deben ser configurables desde la base de datos sin necesidad de recompilar el código del servidor.
RNF-OPE-2	Pila Tecnológica	Para garantizar la homogeneidad y mantenibilidad del sistema, el back-end debe desarrollarse estrictamente en Kotlin con Spring Boot (exponiendo API REST) y el front-end móvil en React Native con Expo. La persistencia de datos relacionales recaerá exclusivamente en PostgreSQL, mientras que la persistencia de datos no relacionales recaerá en un servicio Object Storage - S3 implementado con MinIO.
RNF-OPE-3	Contenerización	El entorno del servidor debe estar empaquetado mediante contenedores de Docker para asegurar que la aplicación funcione de manera idéntica tanto en la máquina local de desarrollo como en el servidor de producción.
RNF-DES-1	Desarrollo	Control de Versiones: Todo el código fuente del proyecto (front-end, back-end y scripts de base de datos) debe estar centralizado y versionado utilizando Git, alojado en un repositorio en la nube como GitHub.

Continúa en la página siguiente

Cuadro 3.3 – continuación de la página anterior

Código	Tipo	Descripción
RNF-DES-2	Desarrollo	Trunk-Based Development (Etiquetas): El equipo de desarrollo debe seguir un flujo de trabajo estructurado. Existirá una rama 'main' que contendrá todo el código desarrollado. Sobre esta rama 'main' se irán creando etiquetas para marcar versiones funcionales y desplegables del sistema.
RNF-DES-3	Desarrollo	Pruebas Automatizadas (Pruebas Unitarias): La lógica de negocio crítica en Kotlin (por ejemplo, el cálculo de los puntos de la liga o la máquina de estados de los partidos) debe estar cubierta por pruebas unitarias independientes de la base de datos.
RNF-DES-4	Desarrollo	Pruebas Automatizadas (Pruebas de Integración): Se deben incluir pruebas que verifiquen el correcto funcionamiento de los endpoints principales y sus consultas contra una base de datos de prueba (por ejemplo, usando "Testcontainers." una base de datos en memoria).
RNF-DES-5	Desarrollo	Gestión de variables de entorno: Ninguna credencial, token secreto o contraseña de base de datos debe subirse al repositorio de código. Todas las configuraciones sensibles deben inyectarse mediante variables de entorno (archivos .env en React Native y application.yml parametrizado en Spring Boot).
Continúa en la página siguiente		

Cuadro 3.3 – continuación de la página anterior

Código	Tipo	Descripción
RNF-DES-6	Desarrollo	El diseño de la API REST debe estar documentado mediante el estándar Open API (Swagger), actuando como único contrato y fuente de la verdad para sincronizar el trabajo entre el front-end y el back-end.
RNF-DES-7	Desarrollo	El código fuente del back-end debe seguir los principios de separación por capas (Controlador, Servicio, Repositorio) para asegurar que la lógica de negocio esté aislada y sea fácilmente verificable. Además, debe aplicarse una "separación vertical" por entidades para mantener dichas funcionalidades claramente separadas.

3.2 Diseño

Definiremos el diseño en dos niveles distintos: la distribución e interacción de los distintos componentes y cómo se va a diseñar cada componente a nivel interno.

3.2.1 Arquitectura del sistema

Las Figuras I.1, I.2, I.3 y I.4 representan los tres primeros niveles de del modelo C4 y explican en detalle la arquitectura establecida para Sports League.

El nivel 1 muestra cómo interactúan los distintos tipos de usuarios con la aplicación. Además, muestra la relación del servicio de autenticación de la UIB con nuestro sistema.

En el nivel 2 podemos observar cómo los distintos componentes interactúan entre ellos: la interfaz gráfica se comunica con el servidor back-end para la consulta y envío de datos a través de peticiones HTTP, mientras que el servidor back-end se comunica con los distintos almacenamientos a través de sus API específicas.

El nivel 3 indica cómo se ha estructurado el back-end: el núcleo del sistema. Podemos observar dos versiones diferentes para este nivel. Para poder cumplir con el requisito no funcional [RNF-DES-7] se ha decidido utilizar dos tipos de arquitectura de software diferentes: una arquitectura de tres capas simple para los módulos que engloban entidades con ciclos de vida simple (creación, visualización y actualización); mientras que para módulos de entidades con ciclos de vida más complejos, (seguridad, autenticación, mantenimiento de sub recursos, cambios y actualizaciones complejas

de varios pasos transaccionales, etc.) se aplica la arquitectura hexagonal o arquitectura de puertos y adaptadores.

En el caso de la arquitectura de tres capas, vemos como hay un flujo de datos desde los controladores que reciben las peticiones del front-end aceptadas por el filtro de seguridad y autenticación, pasando por los servicios sobre los que se delega la lógica de negocio y acabando en los repositorios, encargados de la persistencia y obtención de los datos.

Por otro lado, en la arquitectura hexagonal, vemos como las peticiones llegan a la capa de infraestructura, pasando del filtro de seguridad al adaptador de entrada correspondiente, este procede a llamar al puerto necesario para delegar la lógica de negocio a la capa de aplicación y dicho caso de uso utiliza los repositorios implementados por los adaptadores de salida para comunicarse con las bases de persistencia de datos.

3.2.2 Comportamiento del sistema

Las Figuras I.5, I.6 muestran mediante diagramas usuario-sistema como se deben comportar los diferentes componentes del sistema para poder cumplir con los casos de uso especificado.

El diagrama de la Figura I.5 muestra como la interfaz envía las credenciales al servidor, el servidor las compara con las credenciales existentes en la Base de Datos (BD). Tanto si las credenciales no existen como si la contraseña recibida y almacenada difieren, se deniega el acceso al solicitante. En cambio, si las credenciales existen y las contraseñas coinciden se le permite el acceso al usuario que lo solicita.

El diagrama de la Figura I.6 explica como la interfaz hace una petición al servidor para crear enfrentamientos entre los equipos de una fase. A su vez, el servidor pide la lista de equipos presentes en la fase a la base de datos y esta le devuelve la lista. Si no hay suficientes equipos, se cancela la petición y se devuelve el error a la interfaz; en cambio, si hay suficientes, el servidor aplica el algoritmo de *Doble Round Robin* [5] para la lista de equipos generando los enfrentamientos correspondientes. El servidor guarda los enfrentamientos generados en la base de datos y se devuelve el calendario de enfrentamientos a la interfaz.

3.2.3 Diseño de la base de datos

El diseño de la BD ha sido un derivado directo de los requisitos funcionales y muestra cómo las entidades se relacionan entre ellas a partir de los requisitos. Se ha utilizado la herramienta de draw.io para crear un diagrama de clases UML que muestra la estructura de la BD. En el enlace Diagrama UML - draw.io podemos ver el diagrama más detalladamente.

En el diagrama se pueden apreciar: las clases definidas con sus atributos correspondientes y las relaciones que relacionan dichas clases. Cabe aclarar que se pueden ver algunos de los nombres de clases repetidos junto a un asterisco '*'. Dicho formato indica que la clase ya ha sido definida y que simplemente se usa como una 'referencia'. Se ha decidido aplicar este formato por claridad visual. De esta forma evitamos que las relaciones que conectan dos clases muy separadas no estorben en la lectura del diagrama.

3. ANÁLISIS Y DISEÑO DEL SISTEMA

Aparte de la vista general del diagrama resultaría interesante destacar dos zonas en concreto: una por ser la pieza fundamental del sistema y la otra por resultar en un ejemplo de normalización muy claro.

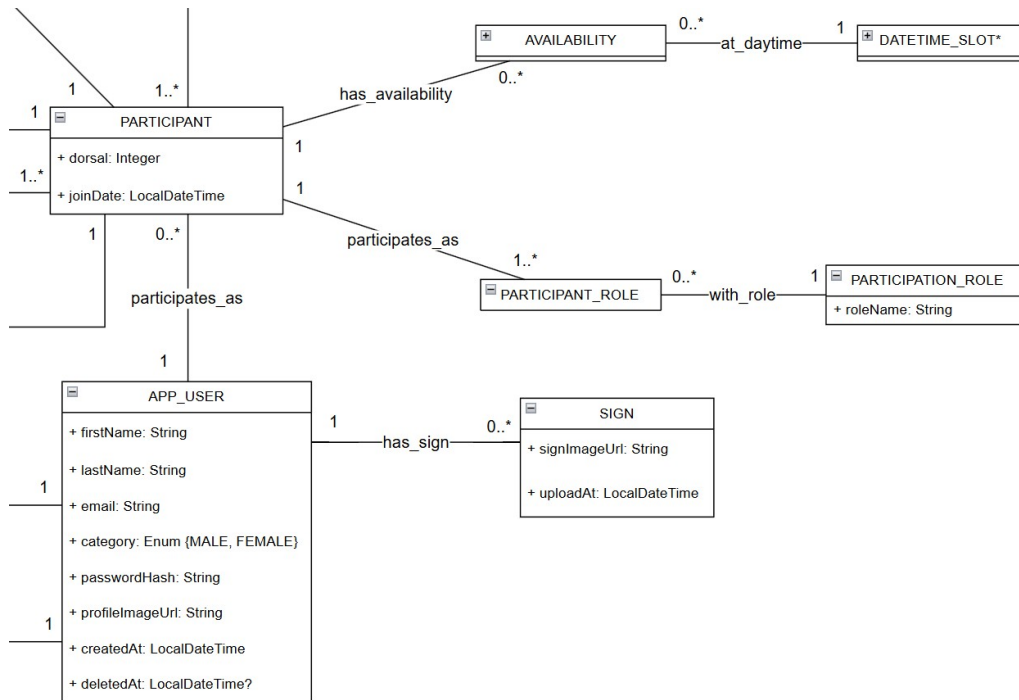


Figura 3.14: Clase participante

En la primera zona (Ver Figura 3.14), la pieza central del sistema, se aprecia la clase Participante: una clase asociativa que relaciona de manera única un usuario con una liga. Esta clase existe porque un usuario puede estar inscrito en varias ligas a la vez y una misma liga puede tener diversos usuarios simultáneos. Esta clase también se relaciona con otra clase asociativa que es la que permite establecer el sistema de roles dentro de una liga. Gracias a estas relaciones conseguimos que un mismo usuario tenga distintos roles dependiendo de la liga. Y, este punto, es el que convierte el sistema en uno que necesita manejar la multitenencia. Además, se puede ver la clase *Availability* que es la que permite que los árbitros puedan registrar su disponibilidad para arbitrar y que el sistema asigne los partidos de forma automática.

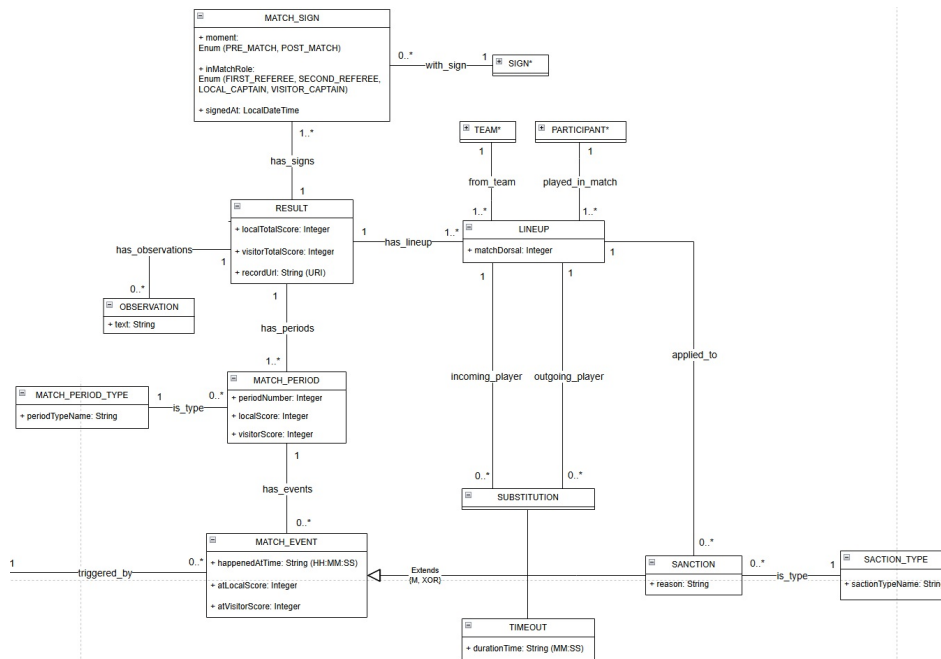


Figura 3.15: Resultado y acta

La segunda zona destacada (Ver Figura 3.15) se relaciona con la parte encargada de gestionar los resultados de los partidos. También, se encarga de la gestión de todos los datos de un partido necesarios para poder generar el acta del partido. Esta zona ha resultado en un buen ejemplo de normalización, ya que, en un principio, se tenía pensado agrupar toda la información del partido en un atributo de la clase Resultado de tipo JSONB.

Se consideró esta primera opción por la naturaleza cambiante y dinámica de la información presente en las actas oficiales de la FVIB (Ver ejemplo en el enlace: Acta oficial). El problema de esta opción era que desnormalizaba en gran manera los datos de un partido, lo cual podría resultar en deuda técnica para manejar dichos datos a futuro. Debido a este problema, se optó por la segunda y opción final: analizar en profundidad todos los aspectos de una acta oficial y modelar estos aspectos manteniendo la tercera forma normal.

Como se puede ver en la Figura [...] tenemos un conjunto de clases relacionadas que modelan todos los datos necesarios para generar un acta oficial. Se podría argumentar que la inclusión de algunos atributos introduce redundancia en el modelo, ya que las puntuaciones de los equipos almacenadas en la clase *Resultado* pueden obtenerse sumando los puntos de los distintos periodos del partido. Sin embargo, se trata de una redundancia controlada y deliberada: los atributos se registran una única vez al cerrar el acta y permiten consultar directamente el resultado global del partido. Esta decisión ofrece dos ventajas. En primer lugar, evita recalcular las puntuaciones cada vez que se consulta el resultado. En segundo lugar, mejora el rendimiento al mostrar listados de partidos, ya que no es necesario sumar los puntos de los periodos de cada partido visible en pantalla.

3.2.4 Diseño del Contrato API y Enfoque *API-First*

El desarrollo del sistema se ha concebido bajo el paradigma de diseño *API-First*. Bajo esta metodología, la interfaz de programación y los acuerdos de intercambio de información no se conciben como un subproducto accidental de la implementación posterior, sino como el paso inicial y vertebrador del diseño del sistema. El contrato resultante actúa como la *única fuente de la verdad* (*Single Source of Truth*) a partir de la cual se deriva la estructura formal de los objetos de transferencia de datos (Data Transfer Object, Objeto de Transferencia de Datos (DTO)), desacoplando de manera efectiva el servicio central de los clientes y garantizando una comunicación consistente y tipada. El código fuente íntegro de la especificación se encuentra formalizado y versionado en su correspondiente repositorio independiente¹.

Para formalizar esta especificación se ha adoptado el estándar abierto **OpenAPI 3.0**, ampliamente consolidado por su neutralidad tecnológica, su riqueza semántica y su capacidad para describir interfaces RESTful de manera exhaustiva.

Modularización y Arquitectura Multidocumento de la Especificación

En un sistema con múltiples dominios de negocio interconectados, agrupar el contrato en un único fichero monolítico compromete la legibilidad y dificulta la mantenibilidad del diseño. Por ello, se ha optado por una arquitectura modular distribuida mediante el uso de referencias canónicas (`$ref`), estructurada en tres capas conceptuales:

- **Fichero Principal (`main.yaml`):** Actúa como punto de entrada del contrato. Define las direcciones base de los entornos de servicio, el esquema de seguridad mediante autorización basada en tokens portadores (`bearerAuth`), el catálogo central de esquemas y el mapeo de rutas de primer nivel.
- **Directorio de Rutas (`paths/`):** Modula las operaciones HTTP (GET, POST, PUT, PATCH, DELETE) agrupadas por área de dominio funcional (`league.yaml`, `match.yaml`, `participant.yaml`, `team.yaml`, `request.yaml`, `user.yaml`, etc.).
- **Directorio de Esquemas (`schemas/`):** Alberga las estructuras de datos de negocio de manera desacoplada de los verbos HTTP, favoreciendo la reutilización transversal de modelos tanto en las peticiones como en las respuestas.

Taxonomía y Ciclo de Vida de los DTOs

El contrato modela las entidades asegurando coherencia con el diseño de la base de datos (ver Sección 3.2.3), pero estableciendo una frontera estricta de encapsulación. Para optimizar el tráfico de red y evitar la exposición innecesaria de atributos internos, se ha diseñado una taxonomía de esquemas clasificada según su propósito funcional:

1. **Esquemas de Creación (`*CreateRequest`):** Definen exclusivamente los parámetros iniciales requeridos para dar de alta una entidad, omitiendo identificadores

¹Repositorio del contrato API: <https://github.com/Carlgm-18/sports-league-api-contracts>

primarios generados por el sistema, marcas temporales de auditoría o estados derivados (e.g., `LeagueCreateRequest`, `PhaseCreateRequest`).

2. **Esquemas de Actualización (*UpdateRequest):** Orientados a modificaciones puntuales o parciales mediante operaciones PATCH o PUT, permitiendo alterar configuraciones o atributos concretos sin obligar al reenvío del objeto completo (e.g., `ConfigurationUpdateRequest`, `ParticipantUpdateRequest`).
3. **Esquemas Resumidos (*Summary):** Representaciones ligeras optimizadas para operaciones de listado, consultas maestras y selectores. Contienen únicamente los atributos visuales clave (e.g., `LeagueSummary`, `ParticipantSummary`), previniendo problemas de sobrecarga en la transferencia de datos (*over-fetching*).
4. **Esquemas Detallados (*Details):** Representaciones completas que encapsulan el grafo exhaustivo del recurso, incluyendo relaciones asociadas, colecciones hijas y estadísticas consolidadas (e.g., `LeagueDetails`, `MatchDetails`, `ResultDetails`).

Complementariamente, los atributos de estado o tipos cerrados se modelan mediante enumerados estrictos (enum), garantizando un vocabulario controlado y libre de inconsistencias para conceptos como el estado del partido (`MatchState`: `NOT_SCHEDULED`, `PENDING`, `ENDED`), el estado de solicitudes (`RequestState`: `PENDING`, `ACCEPTED`, `REJECTED`) o el rol de los colegiados (`RefereeType`: `FIRST`, `SECOND`).

Estrategia de Enrutamiento Híbrida: Jerárquica frente a Plana

Para el diseño de las URIs se ha establecido un patrón híbrido que combina la contextualización de las entidades dependientes con la sencillez en el acceso directo a recursos ya creados:

Tal como se resume en la Tabla 3.4, para las acciones de creación e inscripción se opta por un *enrutamiento jerárquico*. Por ejemplo, al emitir una solicitud para inscribir un equipo o solicitar rol de árbitro, el endpoint se diseña como `POST /leagues/{leagueId}/requests`, ya que el contexto de la liga es indispensable para delimitar la petición.

Por el contrario, cuando un recurso ya cuenta con un identificador único en el sistema, las acciones de consulta, actualización o eliminación adoptan un *enrutamiento plano*. Rutas como consultar la ficha de un club (`GET /teams/{teamId}`), resolver una solicitud (`PATCH /requests/{requestId}`), editar un dorsal (`PATCH /participants/{participantId}`) o fijar las horas de juego (`GET /rounds/{roundId}/availability`) son accesibles de forma directa sin requerir la cadena completa de recursos contenedores.

Polimorfismo en Solicitudes y Respuestas Normalizadas

El diseño contempla asimismo patrones avanzados para satisfacer requisitos funcionales heterogéneos:

- **Polimorfismo Mediante Discriminador:** La gestión de solicitudes unifica distintos trámites bajo una abstracción común (`BaseRequest`). Mediante la propiedad `discriminator` asociada al campo `requestType`, el endpoint `POST`

Patrón de Ruta	Ejemplo Representativo	Propósito y Justificación de Diseño
Jerárquico	POST /leagues/{id}/requests	Creación contextualizada: El nuevo recurso nace subordinado a una entidad superior (la liga), acotando la validez y los permisos de la operación. Facilita la comprensión del alcance de pertenencia directamente desde la propia URI.
(<i>Deep Routing</i>)	POST /leagues/{id}/phases	
Plano o Directo	GET /teams/{teamId}	Operaciones sobre entidades existentes: Una vez que el recurso posee un identificador global unívoco, se prescinde de prefijos padre. Se evita el problema de URLs innecesariamente anidadas (<i>nested URL hell</i>), simplificando las llamadas desde el cliente. Reduce redundancias semánticas en la especificación.
(<i>Flat Routing</i>)	PATCH /matches/{matchId}	
	DELETE /teams/{teamId}	

Cuadro 3.4: Comparativa de patrones de enrutamiento aplicados en el diseño de la API.

/leagues/{leagueId}/requests permite canalizar peticiones de diversa naturaleza (unión a equipo, fundación de equipo o postulación a árbitro), cada una con sus atributos requeridos pero compartiendo un ciclo de resolución homogéneo (ResolveRequestInput).

- **Estandarización Semántica de Respuestas:** Se formaliza el uso riguroso de los códigos de estado HTTP para informar del resultado de cada interacción: códigos 200 OK y 201 Created para resoluciones exitosas, 204 No Content para eliminaciones o actualizaciones sin cuerpo de retorno, y esquemas tipados como FormErrorResponse acompañados de códigos 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found y 409 Conflict para la gestión estructurada de anomalías de negocio.

3.2.5 Diseño de la interfaz

Es necesario pensar en cómo los usuarios pueden interactuar con el sistema para cumplir con requisitos, es decir, necesitamos definir una User Experience, Experiencia de Usuario (UX) que sea, según las respuestas de los usuarios y requisitos funcionales: manejable y fácil de utilizar.

Para el diseño de la interfaz se ha optado por usar la herramienta de Figma al ser un estándar en el mundo del diseño sin coste. La versión preliminar de la interfaz se puede ver a través del siguiente enlace: [Mockup en Figma](#)

Cabe tener en cuenta que este diseño tan solo muestra la forma en la que los usuarios interactuarán con la interfaz y cómo se les mostrará la información. Todo lo relacionado con el diseño visual, estilos, paletas de colores, logotipos y demás se

delegará a un agente de generación de código que será el encargado de darle estilo la interfaz.

Módulo de Arbitraje y Cierre de Acta en Tiempo Real (partido-en-vivo-arbitro)

La pantalla de arbitraje en tiempo real representa el componente de mayor densidad funcional y complejidad técnica de la capa de presentación. Su objetivo fundamental es erradicar por completo el uso del acta física tradicional en papel, implementando una digitalización nativa de extremo a extremo integrada con la lógica de negocio y las políticas de auditoría del back-end.

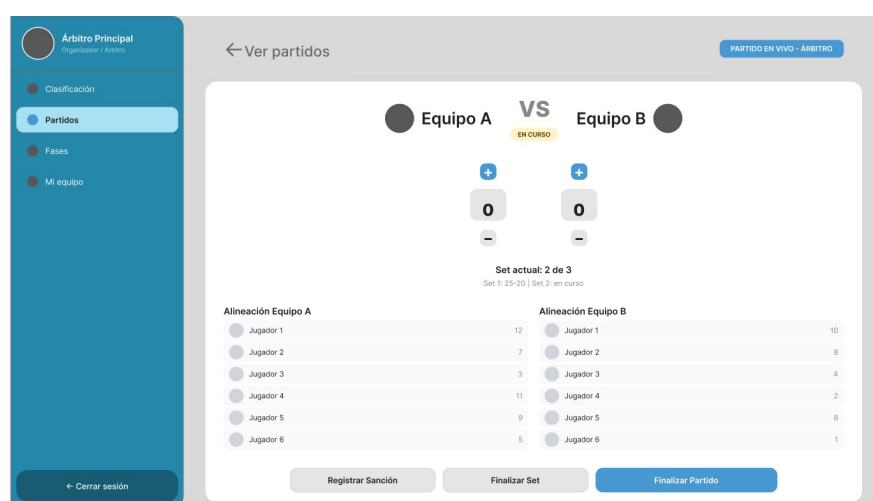


Figura 3.16: Diseño en Figma de la interfaz de arbitraje interactivo y acta en vivo.

Arquitectura y Componentes de la Interfaz

Tal como se ilustra en el diseño de la interfaz (Figura 3.16), la pantalla se articula en torno a cuatro bloques funcionales:

1. **Cabecera y Marcador Táctil Reactivo:** Presenta el emparejamiento en curso (*Equipo Local vs. Equipo Visitante*) mediante un marcador de alta legibilidad diseñado para su manipulación ágil a pie de pista. Incluye controles táctiles de incremento y decremento (+ / -), el cómputo de sets o periodos disputados (e.g., *Set actual: 2 de 3*) y el desglose de los parciales ya concluidos.
2. **Alineaciones y Gestión de Convocatoria:** Permite verificar la lista nominal de jugadores convocados y sus respectivos dorsales oficiales por cada contendiente (entidades Lineup). Facilita al colegiado contrastar las fichas federativas y articular el registro de sustituciones en vivo identificando inequívocamente al jugador saliente y al entrante.
3. **Módulo de Sanciones e Incidencias en Vivo:** A través de una ventana modal desacoplada, el árbitro anota en vivo las incidencias disciplinarias tipificadas

(SanctionType: amonestaciones, tarjetas rojas o faltas reglamentarias) y tiempos muertos (Timeout). Cada evento queda indexado con la marca de tiempo de juego (happenedAtTime), el tanteo en ese momento exacto, el infractor y una observación explicativa textual.

4. **Protocolo de Doble Rúbrica Digital Descentralizada:** En lugar de requerir firmas sobre papel o garabatos en una misma pantalla compartida, el sistema implementa un mecanismo de firma asíncrono y auditado. El reglamento contempla dos momentos clave:
 - **Conformidad Inicial (PRE_MATCH):** Previo al pitido inicial, los capitanes y árbitros validan la convocatoria y el estado de la pista pulsando el botón *Firmar Inicio*.
 - **Ratificación Final (POST_MATCH):** Tras finalizar el partido y revisar los parciales y sanciones, cada actor pulsa *Firmar Cierre* para dar su conformidad al resultado.

Cada acción despacha una petición al back-end que registra una tupla en la tabla MATCH_SIGN, vinculando el instante preciso (signedAt), el momento reglamentario y el rol del firmante (InMatchRole: árbitro principal, segundo árbitro, capitán local o capitán visitante) haciendo uso de la firma digital almacenada previamente en el perfil del usuario.

Flujo de Datos y Consolidación Paperless en el back-end

Desde el punto de vista del ciclo de vida de los datos, la interfaz opera bajo un enfoque completamente digital y desacoplado:

- **Persistencia en memoria volátil durante el juego:** La evolución del tanteo, el control del cronómetro y el apilamiento de eventos se gestionan íntegramente en el estado local de React Native. De este modo se anula la latencia de red en pista, se garantiza la operatividad ante microcortes de cobertura y se permite al colegiado rectificar cualquier anotación antes de su formalización definitiva.
- **Volcado canónico del resultado (PUT /matches/{matchId}/result):** Al pulsar *Finalizar Partido*, el cliente transmite el objeto estructurado ResultDetails. Este paquete de datos consolida los puntos globales, el array de periodos (periods) con sus eventos cronometrados y las observaciones arbitrales. El back-end persiste estas entidades en base de datos transaccional y conmuta el estado del partido a ENDED, lo que permite que la vista materializada de la clasificación se actualice y refleje los nuevos puntos de la liga.
- **Datos necesarios para el acta oficial:** Dado que el sistema rechaza el escaneo de documentos físicos externos, el back-end actúa como la única fuente de la verdad (*single source of truth*). Las tablas de resultados, periodos, eventos y firmas digitales almacenan la información necesaria para generar posteriormente el acta en formato oficial reglamentario. La generación y exportación automática del documento PDF queda como una funcionalidad pendiente.

La pantalla de arbitraje en tiempo real representa el componente de mayor densidad funcional y complejidad técnica de la capa de presentación. Su objetivo fundamental es trasladar la labor del cuerpo arbitral desde el acta física tradicional en papel hacia un flujo digital interactivo, auditado e integrado con el back-end del sistema.

3.2.6 Control de versiones

Para gestionar el control de versiones de los componentes del sistema se ha decidido utilizar Git. Es la herramienta que más se usa dentro de la industria y la más fácil de utilizar para un equipo de desarrollo de una sola persona, como es el caso de este proyecto.

Se han creado tres repositorios, uno para cada parte crítica del sistema: back-end, front-end, contrato API. Se ha optado por esta separación, ya que de esta manera separamos el sistema según el ámbito: definición, procesamiento y presentación de los datos. Un aspecto interesante es que el repositorio del contrato API existe como submódulo de los otros dos. Gracias a esto, ambos componentes tienen una vía directa para actualizar el contrato en caso de ser necesario y consultar los cambios que haya efectuado el otro componente. De esta forma, ambos componentes están al día en caso de tener que cambiar el contenido del contrato debido a la ejecución de un plan de mitigación de riesgos.

Asimismo, se ha habilitado un cuarto repositorio destinado al control de versiones de la memoria del proyecto. Este entorno facilita un seguimiento bidireccional continuo: permite al tutor supervisar el progreso de la redacción y ofrece al alumno un canal claro para gestionar las revisiones. En concreto, el flujo de trabajo se ha articulado mediante *issues* de GitHub, lo que posibilita registrar correcciones puntuales, conservar una trazabilidad completa de los cambios solicitados y vincular directamente su resolución al cierre de cada incidencia a través de los correspondientes *commits*.

Por otra parte, se ha adoptado la convención de *Conventional Commits* para la redacción de los mensajes asociados a cada confirmación, facilitando una lectura más clara del historial del proyecto y simplificando la generación estructurada del registro de cambios.

TECNOLOGÍAS USADAS

Para implementar este tipo de sistemas requerimos de tecnologías que nos permitan cumplir con todos los requisitos especificados y el diseño mostrado en el capítulo anterior. Para cada componente veremos: una breve explicación de la necesidad de dicho componente, listado de las distintas alternativas ideales que existen para dicho componente, una tabla comparativa para distintos aspectos técnicos y la tecnología seleccionada con su correspondiente justificación.

4.1 Servidor back-end

Para el servidor central que gestiona la lógica de negocio y expone la API REST, se requiere un sistema fuertemente tipado, escalable y con un ecosistema maduro de dependencias.

Evaluación de alternativas

Spring Boot (Kotlin)

Framework empresarial maduro que destaca por su robusto sistema de inyección de dependencias y módulos integrados (Spring Security, Data JPA). El uso del lenguaje Kotlin aporta un tipado estático moderno, inmutabilidad y seguridad contra referencias nulas (*Null-safety*) de forma nativa.

Node.js (NestJS)

Entorno de ejecución asíncrono basado en TypeScript. Ofrece tiempos de arranque muy rápidos y un ecosistema masivo de paquetes. Su arquitectura basada en eventos es ideal para alta concurrencia de entrada/salida, aunque delega gran parte de la estructura empresarial a librerías de terceros.

Ktor

Framework asíncrono ligero creado por JetBrains para Kotlin. Ofrece un alto rendimiento mediante el uso de corrutinas y un diseño altamente modular, pero carece de las soluciones empresariales completas que proporciona el ecosistema Spring.

Comparativa técnica

En la Tabla 4.1 se resumen las características fundamentales de cada entorno.

Tecnología Criterio	Spring (Kotlin)	Boot	NestJS (TypeS- cript)	Ktor (Kotlin)
Paradigma	Multihilo / Síncrono		Basado en eventos	Asíncrono (Corrutinas)
Seguridad de tipos	Estricta (Nativa)		Alta (Transpilado)	Estricta (Nativa)
Ecosistema	Muy maduro		Masivo (NPM)	En crecimiento
Arquitectura	Altamente estructurada		Modular	Minimalista

Cuadro 4.1: Comparativa de frameworks para el desarrollo del back-end.

Justificación de la elección

Se ha seleccionado **Spring Boot con Kotlin** por los siguientes motivos técnicos:

- **Diseño arquitectónico:** Su robusta inyección de dependencias facilita la implementación estricta de la Arquitectura Hexagonal, permitiendo un desacoplamiento real entre los puertos y sus adaptadores.
- **Mitigación de riesgos:** Contar con experiencia previa en el ecosistema Spring reduce los tiempos de configuración y asegura una implementación estable del sistema de seguridad e interacciones con la base de datos.
- **Modernización técnica:** La adopción de Kotlin representa un reto académico de alto valor, permitiendo escribir un código de dominio más conciso y expresivo frente a lenguajes tradicionales como Java.

4.2 Base de datos

Para dar soporte a la persistencia del sistema se requiere un Sistema de Gestión de Bases de Datos Relacionales (SGBDR) que garantice el cumplimiento de las propiedades Atomicidad, Consistencia, Aislamiento y Durabilidad (ACID).

Evaluación de alternativas

PostgreSQL

Sistema objeto-relacional de código abierto con adherencia estricta a los estándares SQL. Destaca por su alta extensibilidad, soportando tipos de datos avanzados y manipulación nativa de estructuras semiestructuradas mediante JSONB. Su modelo de

control de concurrencia multiversión (MVCC) optimiza el rendimiento en entornos de alta carga transaccional.

MySQL

Uno de los motores relacionales más extendidos, basado en el motor de almacenamiento *InnoDB* para garantizar soporte ACID. Sobresale por su facilidad de despliegue, curva de aprendizaje inicial baja y alto rendimiento en operaciones de lectura simples.

MariaDB

Bifurcación comunitaria de MySQL, compatible a nivel de binarios, pero con optimizadores de consultas más avanzados y un pool de hilos nativo para mejorar el manejo de la concurrencia.

Comparativa técnica

En la Tabla 4.2 se resumen las principales características técnicas evaluadas.

Tecnología Criterio	PostgreSQL	MySQL (InnoDB)	MariaDB
Modelo	Objeto-relacional	Relacional	Relacional
Cumplimiento SQL	Estricto	Medio-alto	Medio-alto
Consultas complejas	Excelente (CTE, sub consultas)	Adecuado	Bueno
Tipos avanzados	Arrays, JSONB indexable, UUID	JSON básico	JSON básico

Cuadro 4.2: Comparativa de motores de bases de datos relacionales.

Justificación de la elección

El modelo de datos del dominio deportivo presenta una fuerte necesidad de integridad referencial, lo que descarta arquitecturas no relacionales. Se ha seleccionado **PostgreSQL** por los siguientes motivos:

- **Integridad de datos:** Permite aplicar restricciones complejas directamente en la base de datos, asegurando la consistencia del dominio.
- **Flexibilidad estructural:** El uso de JSONB permite almacenar metadatos variables sin perder la rigidez relacional del resto del esquema.
- **Robustez a largo plazo:** Garantiza un rendimiento superior en consultas analíticas complejas (como la generación de clasificaciones o estadísticas).

Para gestionar la evolución del esquema y asegurar la reproducibilidad del entorno, se ha integrado *Flyway*, herramienta que permite aplicar un control de versiones estricto sobre las migraciones *.sql*.

Cabe comentar que para proyectos desarrollados con Spring Boot las alternativas principales son *Flyway* y *Liquibase*. Se ha optado por la primera y no por la segunda, ya que *Liquibase* utiliza formatos como XML, YAML o JSON para definir los cambios, abstrayendo el motor de la base de datos y añadiendo una curva de aprendizaje. Esto sería positivo si se planease cambiar el motor de la base de datos en un futuro, pero para una implementación relativamente sencilla *Flyway* es la alternativa óptima.

4.3 Almacenamiento de objetos

Para la gestión de archivos no estructurados generados por el sistema, como imágenes de perfil o documentos PDF (actas deportivas), se requiere un servicio de almacenamiento de objetos independiente de la base de datos para no degradar el rendimiento transaccional.

Evaluación de alternativas

MinIO

Servidor de almacenamiento de objetos de alto rendimiento, 100 % de código abierto, diseñado para infraestructuras locales o privadas. Es totalmente compatible con la API de Amazon S3, permitiendo una integración estandarizada.

Amazon S3

El estándar de facto en la industria para el almacenamiento de objetos en la nube pública. Ofrece una disponibilidad hasta del 99.999999999 % y escalabilidad ilimitada, delegando la gestión de la infraestructura al proveedor.

Firebase Storage

Servicio gestionado de Google orientado a integraciones rápidas en aplicaciones móviles y web, fuertemente acoplado al ecosistema de Firebase y a sus reglas de seguridad.

Comparativa técnica

En la Tabla 4.3 se analizan las soluciones basándonos en las necesidades del entorno de desarrollo y producción del proyecto.

Tecnología Criterio	MinIO	Amazon S3	Firebase Storage
Despliegue	Local / Autoalojado (Docker)	Nube gestionada (AWS)	Nube gestionada (GCP)
Compatibilidad API	Estándar S3	Estándar S3	Propietaria
Soberanía de datos	Total (Bajo control propio)	Cedida al proveedor	Cedida al proveedor
Gestión de Infraestructura	Responsabilidad del desarrollador	Automática	Automática

Cuadro 4.3: Comparativa de soluciones de almacenamiento de objetos.

Justificación de la elección

Se ha seleccionado **MinIO** como motor de almacenamiento de objetos por las siguientes razones estratégicas:

- **Soberanía y control:** Al desplegarse mediante contenedores de forma local, garantiza la privacidad total de los documentos y evita la dependencia (vendor lock-in) frente a los cambios de políticas de precios de terceros.
- **Entorno de desarrollo aislado:** Permite a cualquier desarrollador levantar el ecosistema completo sin necesidad de configurar credenciales en la nube ni incurrir en costes operativos durante la fase de creación.
- **Compatibilidad futura:** Al implementar exactamente la misma API que Amazon S3, el código desarrollado en el back-end es completamente agnóstico. Si en el futuro el proyecto requiere escalar a la nube pública, la migración a AWS S3 se podría realizar modificando únicamente variables de entorno, sin reescribir la lógica de la aplicación.

4.4 Aplicación front-end

Para la interfaz de usuario, se requiere una solución multiplataforma que evite la duplicidad de bases de código para web, iOS y Android, garantizando un rendimiento fluido y un acceso estandarizado a las funcionalidades nativas del dispositivo.

Evaluación de alternativas

React Native con Expo

Framework que renderiza componentes nativos reales comunicándose a través de un puente con el hilo de JavaScript/TypeScript. Expo abstrae la configuración de los entornos nativos, ofreciendo empaquetado rápido, recarga en caliente y gestión simplificada de despliegues.

Flutter

Framework de Google basado en el lenguaje Dart que dibuja su propia interfaz mediante el motor gráfico Skia/Impeller. Ofrece un rendimiento visual sobresaliente en animaciones complejas, pero requiere adoptar un ecosistema completamente distinto.

Ionic / Capacitor

Solución híbrida que ejecuta una aplicación web estándar (HTML/CSS/JS) dentro de un navegador interno (*WebView*) embebido en la aplicación nativa. Destaca por su facilidad de desarrollo web, pero sufre limitaciones de rendimiento en interfaces pesadas.

Comparativa técnica

En la Tabla 4.4 se detalla la comparativa técnica de las tecnologías evaluadas.

Tecnología Criterio	React Native + Expo	Flutter	Ionic
Renderizado UI	Componentes Nativos	Motor gráfico propio	WebView
Lenguaje	TypeScript / JavaScript	Dart	TypeScript / Web
Acceso a Hardware	Mediante puente (JSI)	Integración compilada	Plug-ins (Capacitor)
Desarrollo ágil	Muy alto (Expo)	Alto	Muy alto

Cuadro 4.4: Comparativa de frameworks para el desarrollo cliente.

Justificación de la elección

Se ha seleccionado **React Native junto con Expo** frente a sus alternativas por las siguientes razones:

- **Experiencia Nativa:** Se descarta Ionic para evitar las limitaciones de rendimiento táctil y visual que imponen los *WebViews*, buscando ofrecer una experiencia de usuario genuinamente nativa.
- **Reutilización de paradigmas:** Al poseer experiencia previa en React, se elimina la curva de aprendizaje de un nuevo lenguaje como Dart. Esto permite invertir el tiempo del proyecto en resolver la lógica de negocio y optimizar las peticiones a la API.
- **Aceleración del ciclo de vida:** Las herramientas de empaquetado de Expo facilitan las pruebas continuas en dispositivos físicos mediante códigos QR, sin necesidad de lidiar con las configuraciones locales de Xcode o Android Studio durante el desarrollo.

IMPLEMENTACIÓN

En este capítulo veremos todo el proceso aplicado para implementar los diseños mostrados anteriormente. Veremos qué ha sido necesario hacer para poder construir todos los componentes que forman el sistema: el servidor back-end que hace de API, la BD, el Object Storage, Almacenamiento de Objetos (OS) y la interfaz de usuario.

5.1 Entorno de desarrollo

5.1.1 Herramientas y versiones

Editores de código

Para poder desarrollar el sistema se han utilizado dos editores de código distintos: uno para el front-end y otro para el back-end.

Para el front-end se ha decidido utilizar Visual Studio Code, ya que es uno de los editores más utilizados dentro de la industria. Además, este editor funciona de forma excelente para desarrollar aplicaciones basadas en entornos Java Script gracias a sus numerosas extensiones.

Mientras que para el back-end se ha optado por IntelliJ IDEA, un IDE de nicho cuando se trata de desarrollar sistemas implementados mediante tecnologías basadas en la JDK, como lo es, en este caso, Kotlin. Además, ofrece bastantes herramientas para trabajar de forma sencilla con lenguajes orientados a objetos, desarrollo de pruebas de código y ejecución y depuración de los programas desarrollados.

5.1.2 Ejecución local

El archivo *docker-compose.yaml* se ha utilizado para configurar un entorno de desarrollo aislado y reproducible. Gracias a los contenedores, el sistema puede levantarse con una única orden y sin depender de una configuración manual compleja de cada servicio. El comando principal es:

```
docker-compose up -d
```

Con esta orden se levantan los servicios de PostgreSQL, MinIO y la API del sistema, así como los recursos necesarios para ejecutar la aplicación en local sin interferir con el entorno de producción.

5.1.3 Migraciones

Como ya se ha comentado en capítulos anteriores, para mantener una correcta gestión de migraciones para la base de datos relacional se ha hecho uso de *Flyway*. Este sistema permite controlar la evolución del esquema mediante archivos `.sql` versionados y aplicados en un orden determinista. El principal beneficio es que la base de datos puede recrearse desde cero con el mismo historial de cambios y sin depender de modificaciones manuales sobre un entorno ya existente.

Cada migración recibe un nombre con formato `VX__descripcion`, donde el número indica el orden de ejecución. De este modo, cuando el sistema se inicia por primera vez, Flyway compara el esquema actual con la historia de migraciones y aplica únicamente las pendientes. Si se quiere modificar una estructura ya distribuida, la práctica correcta es crear una nueva migración acumulativa en lugar de alterar una que ya haya sido aplicada en un entorno compartido.

Este enfoque también ayuda a diferenciar entre datos del esquema y datos iniciales del sistema. Los primeros definen la estructura de la Base de Datos; los segundos, como deportes, roles o sistemas de puntuación por defecto, se insertan como contenido de configuración necesaria para que la aplicación funcione con unos valores iniciales coherentes.

5.1.4 Contrato API y generación de DTO

Para la generación de los DTO en todos los servicios se han utilizado herramientas específicas cuya función principal es la generación de clases y tipos a partir de un contrato API.

En el back-end, se ha utilizado la librería *org.openapi.generator* [6], que, para cada esquema del contrato, genera los *data class* o enumerados correspondientes. En el front-end, el paquete *swagger-typescript-api* [7] genera un archivo con el mismo nombre que el contrato, pero con la extensión `.ts`. Gracias a este flujo, ambos lados de la aplicación utilizan las mismas estructuras de mensaje, reduciendo errores por desalineación y facilitando el mantenimiento del sistema.

5.2 Servidor back-end

5.2.1 Estructura

Mediante la Figura 5.1 como el servidor se ha diseñado como un monolito modular, organizando las responsabilidades por dominio y funcionalidad, en lugar de mantener toda la lógica en un único bloque. Cada módulo contiene sus propios controladores, servicios, repositorios y modelos de dominio. La separación vertical por entidad permite que cada pieza del sistema tenga un alcance claro y que el código pueda crecer sin volverse inmanejable.

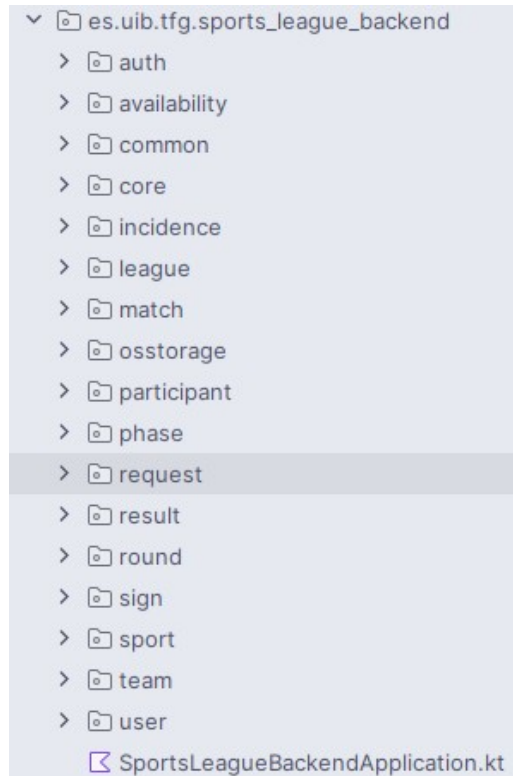
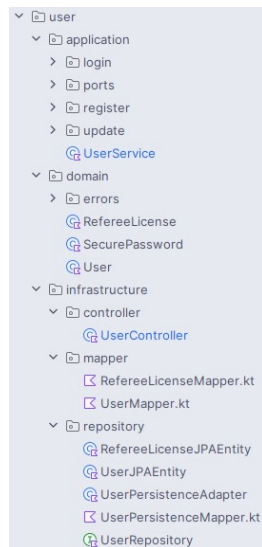


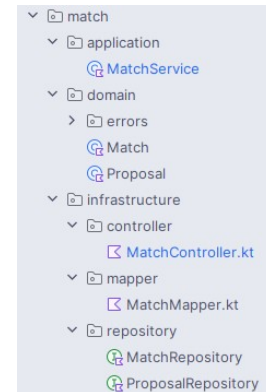
Figura 5.1: Estructura de carpetas

En los módulos con lógica más compleja, como ligas, usuarios y participantes, se ha aplicado una arquitectura hexagonal: el núcleo de negocio queda separado de la infraestructura y de los adaptadores que acceden a la base de datos o a servicios externos. En entidades más simples se ha mantenido una estructura de tres capas, con un controlador, un servicio y un repositorio asociados. En la Figura 5.2 podemos ver la estructura de dos módulos diferentes, cada uno siguiendo una de las arquitecturas explicadas.

5. IMPLEMENTACIÓN



(a) Ejemplo de módulo con arquitectura hexagonal



(b) Ejemplo de módulo con arquitectura de tres capas

Figura 5.2: Ejemplos de arquitecturas en los módulos

5.2.2 Implementación de la seguridad

La seguridad del servidor se ha implementado mediante Spring Security y tokens JSON Web Token (JWT). Cada petición autenticada pasa por un filtro que valida el token e identifica al usuario. A partir de ese punto, la petición se delega a los servicios de pre autorización, que aplican la lógica ABAC necesaria para poder mantener la seguridad y privacidad en los recursos.

Además, se han incorporado validaciones de entrada para evitar que datos mal formados o inconsistentes lleguen al dominio. Esto incluye comprobaciones de campos obligatorios, rangos válidos, tipos esperados y comprobación de permisos antes de ejecutar una operación sensible. Todas estas comprobaciones se hacen de forma automática a través de la anotación `@Valid` incluida en el argumento que representa el cuerpo de la petición para cada endpoint.

5.2.3 Implementación de la lógica de ligas

La lógica de ligas es el eje central del sistema. En ella se gestionan la configuración de cada competición, los equipos, los grupos, las fases, las jornadas y la clasificación. La creación de una liga no se limita a guardar un registro en base de datos: incluye validaciones del formato de competición, compatibilidad con el deporte seleccionado, creación de grupos y generación de los partidos iniciales.

Este diseño permite que cambios en la configuración de una liga se reflejen de forma coherente en toda la estructura del sistema. Por ejemplo, si se modifica el sistema de puntuación, el cálculo de las puntuaciones de los equipos se actualizará, reflejando los cambios en la tabla clasificatoria.

5.2.4 Generación de jornadas y partidos

La generación de partidos se ha automatizado para evitar un proceso manual repetitivo y propenso a errores. El sistema calcula los partidos según: el número de equipos y el tipo de fase. Se aprecian dos procesos distintos dependiendo de si la fase seleccionada es de clasificación o torneo.

Para las fases de clasificación el proceso consta de dos pasos: generación de partidos y generación de jornadas. Tal y como ya se ha visto en la Figura I.6, la generación de partidos se hace mediante la implementación del algoritmo de *Doble Round Robin*. Este algoritmo busca completar el emparejamiento de los equipos de forma equilibrada, alternando las condiciones de local y visitante y evitando que un equipo tenga que jugar más de dos partidos consecutivos como visitante. A continuación, el conjunto de partidos generado se distribuye entre las jornadas disponibles. La asignación se realiza procurando que cada equipo tenga como máximo un partido en cada jornada, de manera que los encuentros de un mismo equipo quedan repartidos a lo largo del calendario. En esta fase todavía no se conocen las fechas y horas definitivas, por lo que no es necesario comprobar solapamientos temporales. Estas se determinan posteriormente mediante el sistema de negociación entre los capitanes, que selecciona una franja compatible con la disponibilidad de los equipos.

Las fases de torneo cuentan con los mismos dos pasos, aunque la generación de los enfrentamientos se realiza de forma diferente. Primero se generan tantas jornadas como niveles tenga el torneo. A continuación, se crean los partidos sin equipos asignados y se identifican mediante índices que representan su posición dentro de un árbol binario. Los enfrentamientos del primer nivel se asignan manualmente. Cuando finaliza uno de estos partidos, su ganador se propaga automáticamente al partido padre correspondiente. Esta relación se calcula mediante la división entera $\lfloor i/2 \rfloor$ sobre el índice del partido hijo, de forma que no es necesario almacenar manualmente todas las relaciones entre rondas. El proceso se repite en los niveles superiores hasta determinar el ganador del torneo.

La ventaja de esta automatización es doble: por un lado, reduce el trabajo manual del administrador; por otro, garantiza que todos los equipos queden cubiertos por un calendario consistente y que las relaciones entre partidos, jornadas y fases se mantengan coherentes dentro de la base de datos.

5.2.5 Gestión de resultados y actas

La gestión de resultados y actas es una de las partes más importantes del sistema, ya que conecta la ejecución del partido con la información definitiva que se muestra a los usuarios. Mientras se realiza un encuentro, el árbitro puede registrar eventos, incidencias, sets, puntos y observaciones del partido. Una vez cerrada el acta, la operación se ejecuta de forma transaccional para actualizar la clasificación y dejar el estado del encuentro en una situación consistente.

Este flujo requiere coordinar varias capas: la validación del estado del partido, la actualización del resultado, la creación del historial de actas y la revisión de la información por parte de capitanes o administradores. El objetivo es que no haya estados intermedios ambiguos que puedan derivar en errores de la competición o en incoherencias en la tabla clasificatoria.

El cierre del acta recibe un objeto `ResultDetails`, cuyos campos se validan antes de persistir el resultado. Esta validación comprueba que la estructura recibida es coherente con el partido y que los periodos y eventos incluidos tienen el formato esperado. Las firmas de capitanes y árbitros siguen un flujo independiente: cada participante firma desde su propio dispositivo y el sistema registra la firma en el momento en que se recibe, sin bloquear el guardado del resultado por la firma pendiente de otro participante. Una vez registrado o modificado un resultado, se actualiza la tabla de clasificación para que refleje el estado más reciente del partido. Esta separación permite corregir posteriormente un resultado erróneo sin perder la posibilidad de mantener la información de las firmas asociada al acta.

5.2.6 Gestión de errores de validación

Las capas de dominio y aplicación gestionan los errores a través del patrón *Result*. Este patrón consiste en la encapsulación de los valores de retorno de las funciones mediante objetos que representan un retorno exitoso o fallido. El resultado, en caso de ser exitoso, contiene el valor retornado; mientras que, si es fallido, contiene el error ocurrido.

El uso de este patrón es preferible frente a usar excepciones, ya que: uno, la generación de excepciones obliga al programa a pausar su ejecución momentáneamente mientras este genera el rastro de la pila para dicha excepción; y dos, a nivel semántico tiene más sentido utilizar errores simples de dominio para expresar fallos de validación esperados, mientras que relegamos el uso de excepciones para estados no esperados del sistema como fallos en la conexión a los diferentes servicios o controladores.

5.2.7 Pruebas y validación

La lógica principal se ha comprobado mediante pruebas unitarias de los cálculos de puntuación y clasificación, la generación de partidos y jornadas, los permisos asociados a los roles y las validaciones y transiciones de estado de los resultados. Estas pruebas permiten verificar la lógica de negocio de forma independiente de los servicios externos y de la persistencia.

También se han implementado pruebas de integración para comprobar los ciclos de vida de una liga, un usuario y un partido. Estas pruebas verifican la interacción entre los endpoints, la lógica de aplicación y la persistencia utilizando definiciones de base de datos en memoria preparadas específicamente para el entorno de pruebas. Además, se ha utilizado una conexión simulada con el almacenamiento de objetos para validar la generación de URLs públicas y de subida.

Finalmente, se ha realizado una validación de extremo a extremo con el arnés de *Antigravity* y el modelo *Gemini 3.5 Flash*, con precisión *Medium*. Esta validación permitió revisar conjuntamente los flujos implementados y comprobar que las diferentes partes de la aplicación funcionaban de forma coherente.

5.3 Base de datos

Como se ha explicado, la BD ha sido implementada con el motor PostgreSQL. Se ha buscado cumplir la tercera forma normal para asegurar la mantenibilidad, unicidad e integridad de los datos en todo momento. La estructura almacena información

de usuarios, ligas, equipos, jugadores, partidos, actas, roles y permisos, todos ellos relacionados mediante claves primarias y foráneas.

La base de datos se ha implementado mediante una serie de migraciones `.sql` cuyo propósito es mantener un historial claro del desarrollo del esquema. En concreto, se ha utilizado un enfoque con cuatro pasos: creación del esquema inicial, instalación de disparadores para borrado lógico, creación de la vista de clasificación y carga de datos iniciales.

Además, se han aplicado restricciones de unicidad para evitar duplicados y relaciones asociativas para modelar relaciones muchos a muchos, como la pertenencia de jugadores a equipos o la participación de equipos en una liga. El uso del borrado lógico garantiza que la información no pueda perderse de forma irreversible si se elimina un recurso que tiene implicaciones históricas o administrativas.

5.4 Almacenamiento de objetos

Durante la inicialización del contexto de Spring Boot, la clase de configuración `MinioConfig` verifica de forma reactiva la existencia del *bucket* configurado (`minio.bucket`) mediante el cliente oficial de MinIO (`client.bucketExists`). Si el contenedor no está presente en la instancia de almacenamiento, se procede a su aprovisionamiento automático (`client.makeBucket`) y se aplica una política de acceso basada en declaraciones (`SetBucketPolicyArgs`). Esta política concede permisos públicos de solo lectura (`s3:GetObject`) sobre los recursos alojados, garantizando la persistencia idónea tanto en el primer despliegue como en reinicios tras fallos del servicio, sin requerir intervención manual por parte de los administradores de infraestructura.

Para la gestión de los objetos, el sistema adopta un esquema de rutas jerárquicas virtuales basadas en prefijos, delimitados por el enumerado de dominio `StorageFolder` (`avatars`, `profiles`, `league-banners`, `team-shields`, `signatures` y `match-records`). Cada fichero se identifica de forma determinista mediante la concatenación de la carpeta correspondiente y un identificador único universal (UUID v4) junto con la extensión indicada ("`$folder/$uuid.$extension`"). Por ejemplo, la firma digitalizada de un usuario se almacenará bajo la clave:

`signatures/550e8400-e29b-41d4-a716-446655440000.png`

Este criterio erradica cualquier riesgo de colisión por coincidencia de nombres entre usuarios, previene ataques de inyección en la ruta (*path traversal*) y asegura que los identificadores de recursos no revelen información secuencial predecible.

La subida de archivos se desacopla del servidor de aplicaciones mediante el patrón de *Presigned URLs* (URLs prefirmadas). En lugar de canalizar pesados flujos binarios multipart a través de la memoria JVM del back-end, el servicio `StorageService` valida previamente la categoría solicitada (`StorageFolder.isValid`) y expide una URL de subida temporal con verbo HTTP PUT firmada criptográficamente, acotada a una ventana de expiración estricta de cinco minutos. La carga efectiva del archivo la efectúa el cliente directamente contra el servidor de almacenamiento. Finalmente, los parámetros de conexión y las credenciales (`endpoint`, `access-key`, `secret-key` y `bucket`) se inyectan dinámicamente desde variables de entorno mediante anotaciones `@Value`,

preservando el principio de doce factores (*12-Factor App*) y asegurando que los secretos del entorno de producción nunca queden expuestos en el código fuente.

5.5 Interfaz multiplataforma

5.5.1 Navegación

La navegación dentro de la interfaz está basado en el componente `Stack` que permite definir las pantallas que la interfaz incluye. Este componente es el que permite la navegación con rutas parametrizadas mediante parámetros URL. Estos parámetros son útiles a la hora de obtener la información de una página en concreto. El principal uso que se hace es el de obtener el identificador de un recurso que se quiere consultar para poder hacer la petición al endpoint correspondiente del back-end.

5.5.2 Comunicación con la API

La comunicación entre la aplicación móvil y la API REST del back-end se articula a través de un cliente HTTP centralizado (`apiClient`), desarrollado de forma ad-hoc sobre la interfaz nativa `fetch` de JavaScript.

A diferencia de los enfoques convencionales que delegan la capa de red en librerías de terceros (como `axios`) o en clientes generados automáticamente por herramientas como `swagger-typescript-api`, se ha optado por una implementación propia y minimalista que desacopla la definición estática de tipos del código de transporte de red. De este modo, los contratos OpenAPI se emplean exclusivamente para la generación de interfaces TypeScript (mediante el flag `-no-client`), garantizando el tipado estricto en tiempo de compilación sin incorporar dependencias externas en tiempo de ejecución.

Esta decisión responde a criterios estrictos de seguridad orientados a mitigar los riesgos asociados a ataques sobre la cadena de suministro (*Supply Chain Attacks*) en el ecosistema *npm*, los cuales se detallan en la sección 6.2.1. Al basarse exclusivamente en APIs nativas del estándar ECMAScript, se reduce al mínimo la superficie de ataque, se evitan árboles de dependencias transitivas innecesarias y se obtiene un control total sobre el ciclo de vida de la petición: desde la inyección automática del token de autenticación JWT en la cabecera `Authorization: Bearer` a partir del almacenamiento seguro del dispositivo, hasta la normalización de las respuestas y la captura homogénea de errores HTTP bajo un tipo unión discriminado (`ApiResponse<T>`).

5.5.3 Estilo de la interfaz

Tal y como se ha explicado en la sección 3.2.5, el diseño visual de la interfaz se ha desarrollado a partir de los prototipos elaborados en Figma y se ha concretado mediante la generación y adaptación de los estilos CSS. Para ello, se ha utilizado el arnés *Antigravity* junto con el modelo *Gemini 3.5 Flash*, configurado con un nivel de precisión *Medium*. Como contexto para la generación, se proporcionaron los prototipos de Figma, los requisitos no funcionales y las directrices relacionadas con la temática deportiva de la aplicación.

El resultado se ha revisado y ajustado para mantener la coherencia visual entre las distintas pantallas, mejorar la legibilidad de la información y adaptar la interfaz a los

diferentes dispositivos y tamaños de pantalla. Las figuras siguientes muestran algunos ejemplos de las pantallas implementadas y permiten comparar la propuesta visual con los diseños definidos durante la fase de prototipado.

5.5.4 Flujos por rol

Tal y como se muestra en el diagrama de actividades de la Figura 5.3, los flujos de trabajo de la aplicación se adaptan a las responsabilidades de cada rol. El administrador configura la competición, crea sus fases y jornadas, genera el calendario y supervisa su evolución. El capitán administra la información de su equipo, consulta los partidos asignados y coordina su disponibilidad. Durante la celebración de un encuentro, el árbitro registra los eventos y resultados, completa el acta y la cierra para que el sistema actualice la clasificación. Finalmente, el jugador puede consultar los próximos partidos, los resultados, la clasificación y la información relacionada con su equipo. Esta distribución de funciones permite limitar las operaciones disponibles según los permisos de cada usuario y ofrece una experiencia adaptada a las necesidades de cada participante.

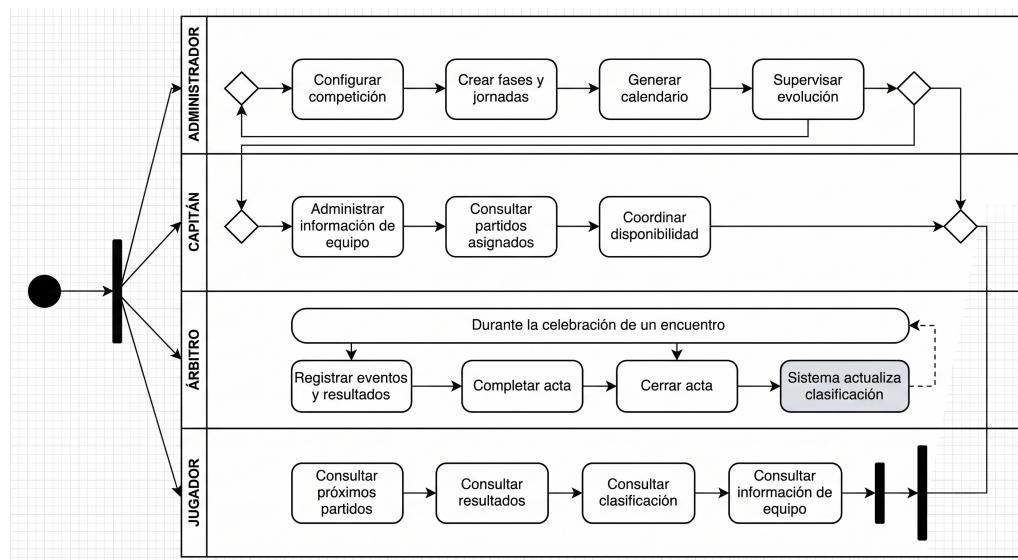


Figura 5.3: Diagrama de actividades del flujo por rol

En resumen, la implementación demuestra que la arquitectura y el diseño planteados en los capítulos previos no son solo una propuesta teórica, sino un sistema funcional con lógica de negocio, persistencia, almacenamiento de archivos y una interfaz de usuario consistente. La combinación de diferentes tecnologías y patrones ha permitido construir una solución que cubre de forma integral la gestión de ligas universitarias, manteniendo una estructura escalable y modular.

CAPÍTULO 6

RESULTADO

En este capítulo se evalúa el grado de cumplimiento de los requisitos y objetivos definidos inicialmente, comparándolos con la solución final implementada. También se indican las desviaciones producidas durante el desarrollo y los elementos que no se han podido verificar de forma completa.

6.1 Cambios aplicados

Respecto a los requisitos iniciales, el sistema cumple las funcionalidades principales de gestión deportiva definidas durante la fase de análisis. Se han implementado el registro y la autenticación propia de usuarios, la creación y configuración de ligas, la gestión de equipos y participantes, el sistema de roles, la creación de fases y rondas, la generación de calendarios, la negociación de fechas, la asignación de árbitros y la gestión de resultados y firmas digitales. La única funcionalidad funcional prevista que no se ha implementado es la autenticación mediante las credenciales institucionales de la UIB.

El desarrollo se organizó en cinco fases consecutivas: análisis, diseño, implementación, pruebas y documentación. Todas las fases se completaron dentro del periodo previsto. Para reducir el impacto de las obligaciones académicas de otras asignaturas y de los periodos de exámenes, se reservó margen de tiempo en la planificación inicial. Esta previsión permitió atender dichas obligaciones sin modificar la duración de las fases ni retrasar la finalización del proyecto.

6.2 Desviaciones respecto a la planificación inicial

6.2.1 Tecnología

Durante el desarrollo del sistema surgieron varios cambios que afectaron al proceso de implementación y a la forma de gestionar la infraestructura.

El principal cambio que afectó al front-end fue la necesidad de cambiar el gestor de dependencias de JavaScript por motivos de seguridad. La aparición de ataques de cadena de suministro [8] contra paquetes populares de *npm* hizo necesario migrar el proyecto a *pnpm*, un gestor que ofrece mejor control sobre las versiones y configuraciones de las dependencias. Esta decisión modificó la configuración inicial, pero no redujo el alcance funcional previsto.

Otra desviación en el front-end ha sido el cambio de sistema para gestionar los estilos de la interfaz. En primera instancia se utilizaba el componente nativo *StyleSheet*. Este componente permite escribir los estilos de una forma similar a CSS puro, pero mediante objetos dentro del mismo archivo *.ts*. En ocasiones, esto resultaba engorroso al querer aplicar estilos relativamente complicados, ya que las funciones nativas de CSS no estaban disponibles al ser solo parte de la API de estilos enfocada a web y no a multiplataforma. Por este motivo se decidió cambiar a *Nativewind*: una librería de estilos que permite utilizar el sistema de estilos de Tailwind CSS en aplicaciones de React Native.

Por otra parte, hubo inconvenientes al querer implementar la verificación de credenciales de la UIB. Tras consultar a los servicios de soporte informático sobre la posibilidad de utilizar alguna herramienta con acceso mediante peticiones HTTPS, se indicó que dicha herramienta no estaba disponible para proyectos desarrollados como TFG. Por este motivo, se aplicó la gestión del riesgo correspondiente y se decidió dejar esta funcionalidad fuera de la versión desarrollada.

El desarrollo se completó dentro del plazo previsto en el cronograma. Las modificaciones introducidas afectaron principalmente a decisiones tecnológicas y no supusieron una reducción general del alcance funcional. La integración con servicios de auditoría y la preparación de un despliegue productivo se mantienen como mejoras futuras, ya que no formaban parte de los requisitos mínimos de la versión desarrollada.

6.3 Funcionalidades implementadas

El sistema implementa completamente la gestión de usuarios, ligas, equipos y participantes, incluyendo sus solicitudes y relaciones. También se han implementado el sistema de roles, la configuración de fases y rondas, la generación de partidos clasificatorios y de torneo, la distribución de partidos entre jornadas, la negociación de fechas, la gestión de disponibilidades y la asignación manual y automática de árbitros.

Asimismo, se ha completado el flujo de resultados y actas digitales. El sistema permite registrar puntuaciones, periodos, eventos, observaciones y firmas digitales de capitanes y árbitros. Al guardar un resultado, la clasificación se actualiza con los datos vigentes del encuentro. La generación automática de un documento oficial de acta no se ha implementado, por lo que constituye la principal funcionalidad funcional pendiente.

Los requisitos no funcionales relacionados con seguridad, fiabilidad, eficiencia de almacenamiento, operación, contenerización, control de versiones, contrato API y pruebas automatizadas se consideran implementados. Las pruebas unitarias, de integración y de extremo a extremo ejecutadas para comprobar el funcionamiento general del sistema fueron satisfactorias. No obstante, no se han verificado experimentalmente

los objetivos de resolver la clasificación en menos de 200 ms ni de mostrar el Dashboard en menos de 3 segundos.

El estado de los elementos que se apartan del cumplimiento completo se resume en la siguiente tabla:

Elemento	Estado	Explicación
Autenticación institucional de la UIB	No implementado	No se pudo acceder al servicio institucional necesario.
Generación automática del acta oficial	Pendiente	Se almacenan los datos, eventos y firmas digitales, pero no se genera el documento oficial.
Tiempo de respuesta de la clasificación	No verificado	No se realizó una medición experimental que confirme el objetivo de 200 ms.
Tiempo de arranque del Dashboard	No verificado	No se realizó una medición experimental que confirme el objetivo de 3 segundos.

Cuadro 6.1: Elementos cuyo cumplimiento no es completo o no ha sido verificado.

De acuerdo con la tabla de requisitos funcionales de la sección 3.1.2, los requisitos funcionales codificados se consideran implementados. La autenticación institucional de la UIB no forma parte de dicha tabla y se trata de una funcionalidad adicional que no pudo incorporarse por la falta de acceso al servicio correspondiente. Del mismo modo, la generación automática del acta oficial aparece como una funcionalidad deseada en los formularios, pero no como un requisito funcional codificado; por tanto, se considera una funcionalidad prevista pendiente y no un incumplimiento de los requisitos funcionales definidos.

6.4 Funcionalidades no implementadas

Las funcionalidades previstas que no se han implementado son la autenticación con credenciales institucionales de la UIB y la generación y exportación automática del acta oficial. La primera no pudo desarrollarse por la falta de acceso al servicio institucional y la segunda quedó fuera de la implementación final, aunque se mantuvo el registro digital de resultados, eventos, firmas y observaciones. La integración de servicios de auditoría y el despliegue en un entorno productivo se consideran mejoras futuras, no incumplimientos de los requisitos funcionales codificados.

Por último, aunque se han ejecutado pruebas unitarias, de integración y de extremo a extremo, el sistema no se ha validado durante una temporada completa de competición. Por ello, los objetivos de rendimiento indicados anteriormente permanecen sin verificar en un escenario real prolongado.

CONCLUSIONES

7.1 Cumplimiento de objetivos

El proyecto cumple de forma sólida los objetivos principales planteados en la introducción. El primer objetivo, la recogida de necesidades de los distintos perfiles implicados, se ha abordado mediante el análisis de usuarios, los formularios y la definición de roles como jugador, capitán, árbitro y administrador. El segundo objetivo, diseñar un sistema que respondiese a esas necesidades, quedó plasmado en la arquitectura del sistema y en la definición de los procesos de liga, partidos, actas y clasificación. El tercero, implementar una solución multiplataforma, se materializó en la combinación de un back-end con Spring Boot, un front-end móvil con React Native y Expo, y un servicio de almacenamiento con MinIO. El cuarto objetivo, preparar un despliegue reproducible, se ha abordado con Docker y con un entorno de desarrollo basado en contenedores, aunque la fase de producción sigue siendo un punto de mejora y consolidación. La implementación de estos objetivos se ha comprobado mediante pruebas unitarias, pruebas de integración de los ciclos de vida principales y una validación de extremo a extremo de la aplicación.

7.2 Resultados obtenidos

El sistema desarrollado ofrece una base funcional para la gestión de ligas universitarias, con capacidades de creación de ligas, gestión de equipos y jugadores, control de participantes, generación de calendarios, cálculo de clasificaciones y gestión de actas. La generación de partidos y jornadas se adapta al formato de la competición, mientras que el cierre de resultados actualiza la clasificación y permite registrar las firmas de los participantes de forma independiente. La solución también permite separar claramente la experiencia de cada perfil de usuario, evitando que un único tipo de usuario sea responsable de toda la operación de la liga. En este sentido, la aplicación no solo automatiza tareas rutinarias, sino que también mejora la trazabilidad y la consistencia

de los datos del sistema.

Aunque el alcance del proyecto no alcanza aún un despliegue completo para producción, sí queda demostrada la viabilidad técnica del enfoque propuesto. La arquitectura cliente-servidor, la base de datos relacional, la integración con MinIO y la interfaz móvil configuran una solución útil y extensible para usos reales dentro del ámbito universitario.

7.3 Mejoras aportadas con respecto a las alternativas

La principal mejora aportada respecto a Winner y Clupik es la gestión centralizada dentro de una única aplicación, con varios perfiles coordinados dentro de la misma liga. Esto evita la fragmentación de herramientas y permite que administradores, capitanes, árbitros y jugadores utilicen un mismo ecosistema para cumplir tareas distintas pero relacionadas entre sí.

Además, el sistema incorpora un mayor nivel de personalización tanto a nivel de equipos como de jugadores, permite negociar las fechas de los partidos, configurar distintas fases competitivas y asignar árbitros. También ofrece un flujo de acta digital con firmas desde los dispositivos de los participantes y permite el autoalojamiento mediante Docker. Estas funcionalidades aparecen como opciones limitadas o de pago en los planes analizados de Clupik y Winner, según la información observada en sus respectivas imágenes de comparación. El uso de un contrato API entre front-end y back-end reduce asimismo la inconsistencia entre capas del sistema.

Otra mejora relevante es la posibilidad de autoalojamiento, ya que el proyecto puede desplegarse en un entorno propio con Docker y servicios locales. Esto aporta un mayor nivel de control sobre los datos y reduce la dependencia de plataformas de terceros, algo especialmente valioso cuando se gestionan ligas académicas o instituciones públicas.

7.4 Limitaciones

El proyecto también presenta limitaciones claras. En primer lugar, la muestra de opinión recogida en los formularios es limitada y no permite validar completamente la utilidad del sistema sobre una temporada real completa. En segundo lugar, no se ha podido completar la integración con la autenticación institucional de la UIB ni la validación de credenciales oficiales, por restricciones de seguridad y alcance. Asimismo, la fase de despliegue en producción sigue siendo limitada y necesitaría ajustes adicionales en autenticación, copias de seguridad, observabilidad y gestión de secretos.

También hay que reconocer que el proyecto no ha sido validado con una temporada completa de competición ni se han medido experimentalmente los objetivos de tiempo de respuesta de la clasificación y de arranque del Dashboard. Esto limita la generalización de algunas conclusiones, aunque no resta valor a la base técnica y funcional que se ha construido.

7.5 Aprendizaje

Gracias a este TFG se han adquirido conocimientos en áreas fundamentales del desarrollo de software moderno. Se ha trabajado con Spring Boot y Kotlin para crear una API

robusta, con React Native y Expo para construir una interfaz móvil multiplataforma, con PostgreSQL para modelar datos relacionales, con Flyway para gestionar evolución del esquema y con MinIO para manejar almacenamiento de objetos. También se ha aprendido a organizar un proyecto con contenedores mediante Docker y a diseñar soluciones de una forma que separa claramente la lógica de negocio de la infraestructura.

El trabajo también ha permitido comprender la importancia de la ingeniería de requisitos, la arquitectura y la validación de decisiones técnicas. En un proyecto de este tipo, una buena definición del problema y un diseño claro son tan importantes como la implementación concreta, ya que condicionan toda la evolución posterior del sistema.

7.6 Trabajo futuro

Como trabajo futuro, la primera prioridad sería la preparación del sistema para un entorno de producción real. Esto incluiría mejorar la seguridad, completar la autenticación institucional, añadir trazabilidad y observabilidad, y reforzar la gestión de copias de seguridad. También sería necesario ampliar la automatización de despliegue y preparar una estrategia clara de mantenimiento y actualización.

Además, se podrían incorporar nuevas funcionalidades como notificaciones push, paneles de administración, integración con más deportes y soporte para otras configuraciones competitivas. Una mejora interesante sería ampliar la lógica de acceso a la liga con criterios configurables por organizador, así como añadir una capa de auditoría más completa para cumplir requisitos de sistemas públicos y de entidades académicas.

En última instancia, el objetivo del proyecto no es solo crear una herramienta específica para una liga concreta, sino dejar una base extensible, reutilizable y sostenible que pueda adaptarse a nuevas necesidades y escenarios deportivos.



FORMULARIOS DE OPINIÓN

RESULTADOS DE LOS FORMULARIOS

B.1 Respuestas a las preguntas abiertas

Otros aspectos

- Me gustaría que hubiera estado más adaptada al voleibol. Se nota que no está hecha para este deporte por el tema de la complicación de asignar una distinta puntuación a diferentes finales de partido o la falta de aparición de los puntos a favor y él contra en la clasificación. Además la asignación de los árbitros a los partidos es muy tediosa por tener que seleccionarlos uno a uno. Me gustaría también que se creará un calendario donde se viera más claramente cuando es cada partido y árbitro asignado.
- Yo solo querría que cada equipo gestionase su plantilla y que los resultados y la clasificación se actualice.
- Que se puedan visualizar los puntos de cada equipo una vez finalizado el partido. Que tenga la opción de notificaciones para los próximos partidos.
- Quiero, que los puntos se repartan eficientemente, porque la aplicación está programada para una liga de fútbol y no para puntualizar marcadores de vóley.
- Quiero que se puedan personalizar los propios equipos porque me parece una buena forma de participación.

Problemas encontrados

- Me lío a veces al poner los resultados de los sets en cada apartado de los partidos.
- La aplicación que hay ahora solo se podía descargar en iPhone o era muy complicada de descargar.

B. RESULTADOS DE LOS FORMULARIOS

- El desplegable de los grupos puede crear confusión, ya que no se modifica cuando has seleccionado otro deporte. Está predeterminado para el de fútbol. También en el apartado para ver los resultados deporte cada jornada (por fechas) solo está disponible los resultados de la primera jornada.
- Hay funciones que no terminan de cargar como la bandeja de entrada o las notificaciones, va bastante lenta también.
- Me cuesta entrar porque no la veo nada práctica ni estética.
- El sistema de puntos suele ir mal, a veces diversos equipos han aparecido con otra cantidad de puntos que les corresponde a otros equipos.
- Hay errores en el tema de los partidos, por ejemplo hay partidos que no me salen de mi equipo o que aparecen como si no hubiesen pasado, pero ya han pasado varias semanas. No sé si es parte de la app o de quien mantiene la puntuación de cada partido, pero igual queda raro.



INTERFAZ WINNER

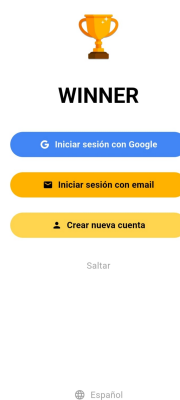


Figura C.1: Registro e inicio de sesión

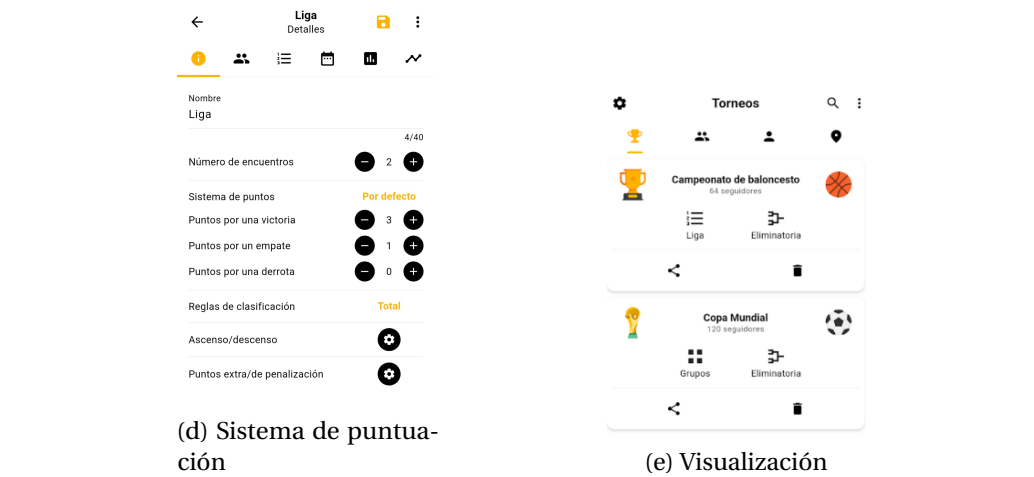
C. INTERFAZ WINNER



(a) Selección de deporte y formato de participación

(b) Deportes disponibles

(c) Personalización de fases



(d) Sistema de puntuación

(e) Visualización

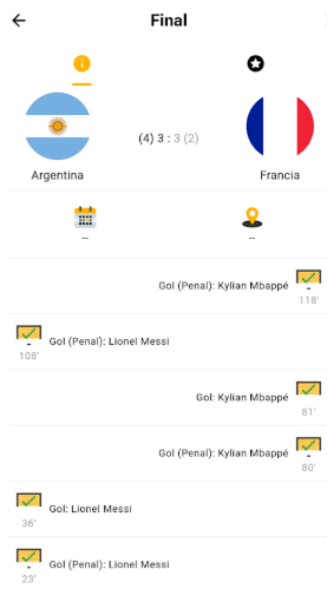


(a) Equipos

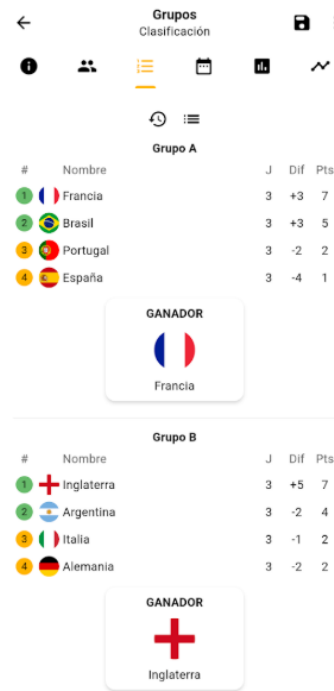
(b) Jugadores

(c) Personalización imagen

(d) Personalización color



(a) Resultado de partidos



(b) Clasificación



(a) Resultado de partidos



(b) Clasificación

Figura C.5



INTERFAZ CLUPIK

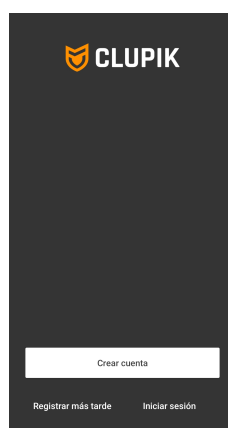
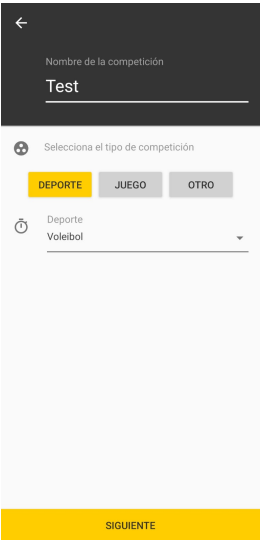
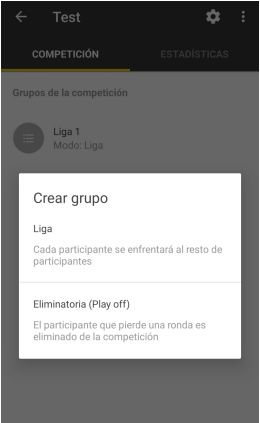


Figura D.1: Registro e inicio de sesión

D. INTERFAZ CLUPIK



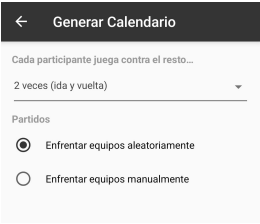
(a) Selección de deporte



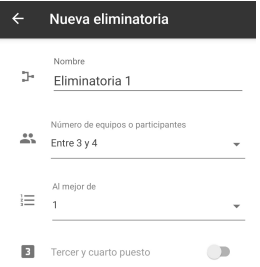
(b) Creación de fase



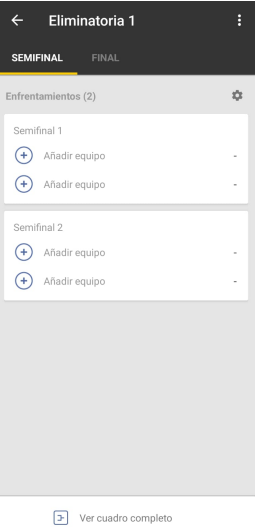
(c) Sistema de puntuación



(d) Fase de clasificación



(e) Fase de torneo



(f) Actualizar enfrentamientos

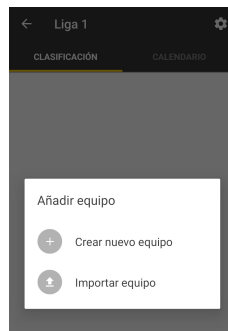
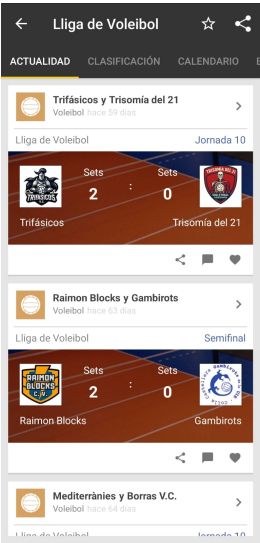
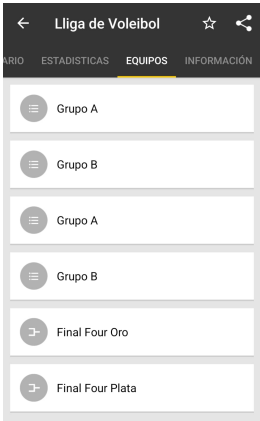


Figura D.3: Creación de equipos y jugadores

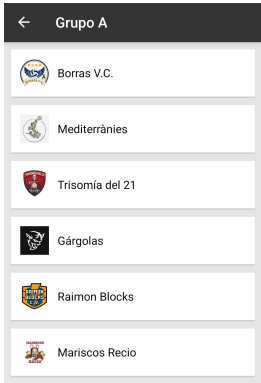
D. INTERFAZ CLUPIK



(a) Partidos disponibles



(b) Grupos



(c) Equipos



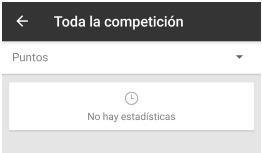
(d) Jornadas



(e) Partidos de la jornada

Grupo B													
Ps	Nombre	P	PJ	PG	PE	PP	F	C	D				
1	Red Birds	30	12	12	0	0	24	3	21				
2	Trifásicos	26	11	9	0	2	20	4	16				
3	Inter Erasmus	19	12	7	0	5	15	11	4				
4	Mopis VC	14	12	4	0	8	12	18	-6				
5	Super Campeones	12	10	5	0	5	10	13	-3				
6	Black Goat	10	12	3	0	9	7	18	-11				
7	Gambirots	1	11	0	0	11	1	22	-21				

(f) Clasificación por grupo



(g) Estadísticas

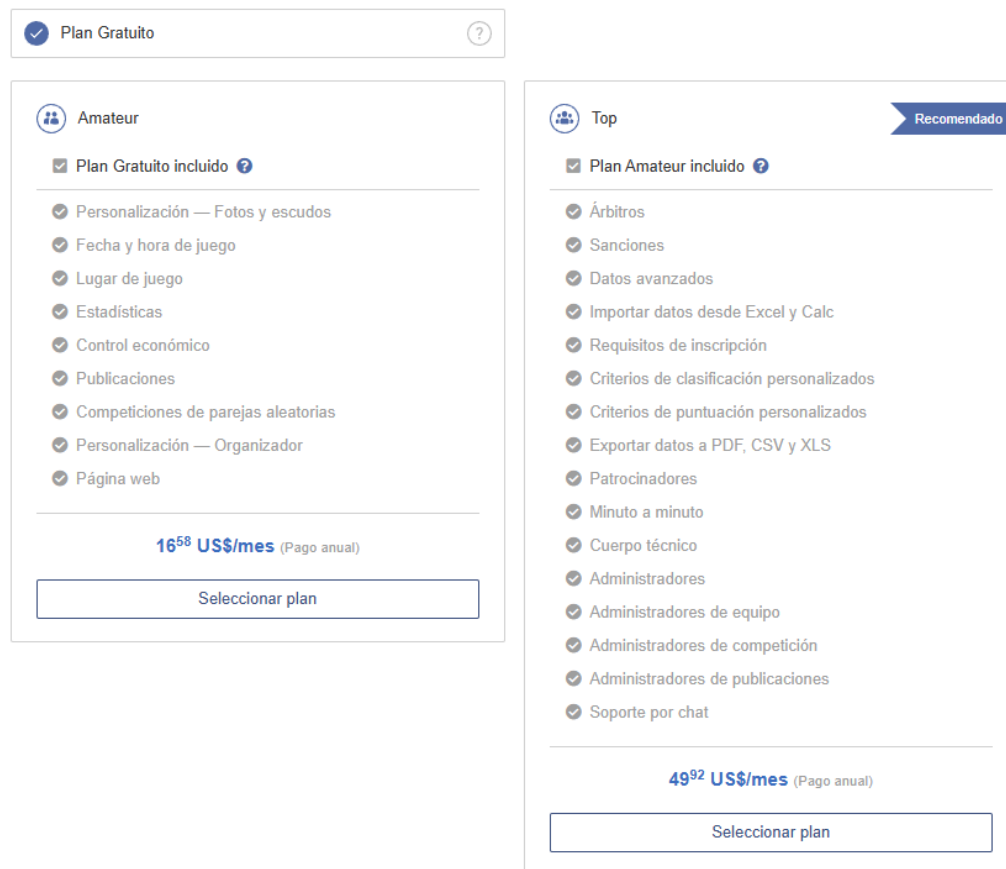
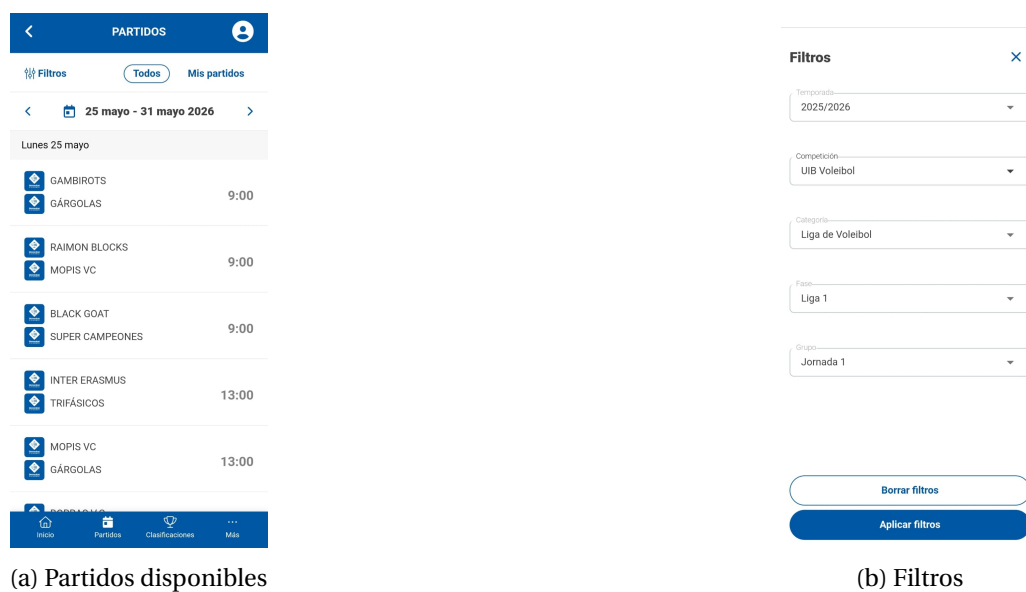


Figura D.5: Planes de pago



(a) Partidos disponibles

(b) Filtros

Figura D.6: Vista desde la aplicación de la UIB

REQUISITOS

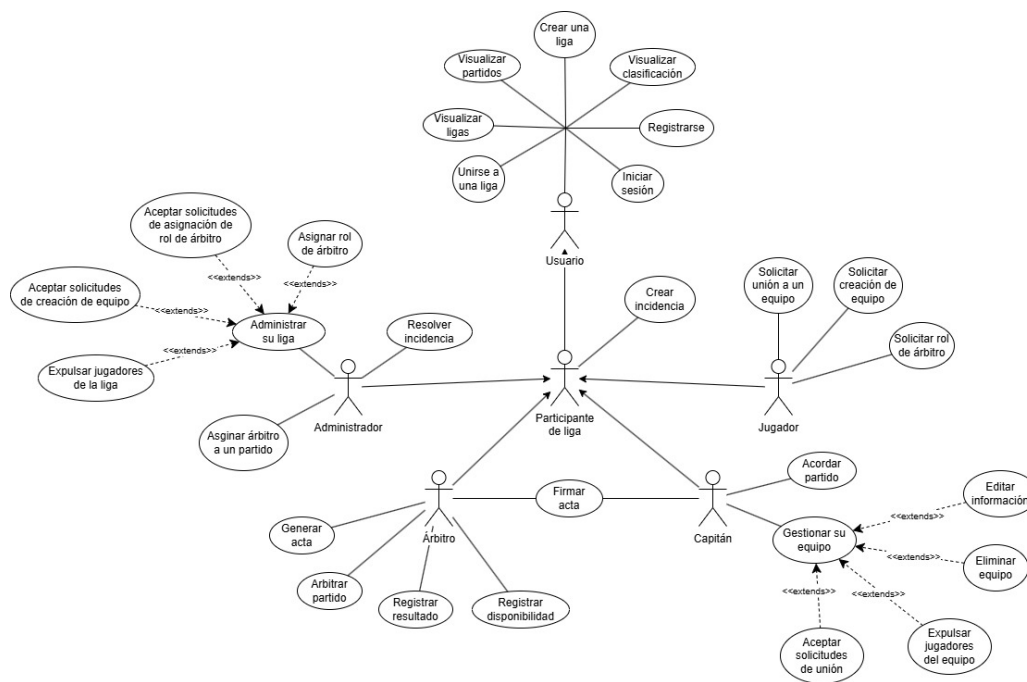


Figura E.1: Diagrama de casos de uso

CRONOGRAMA

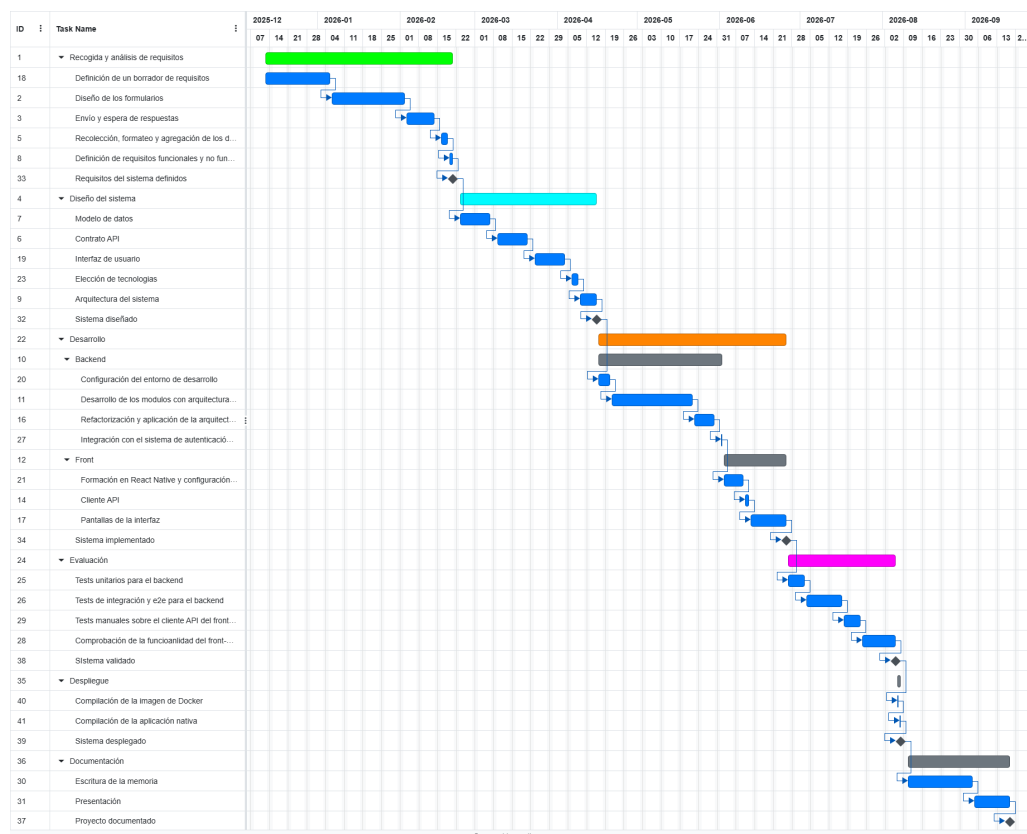


Figura F.1: Cronograma



DISEÑO DE LA BASE DE DATOS



CONTRATO API

DISEÑO DEL SISTEMA

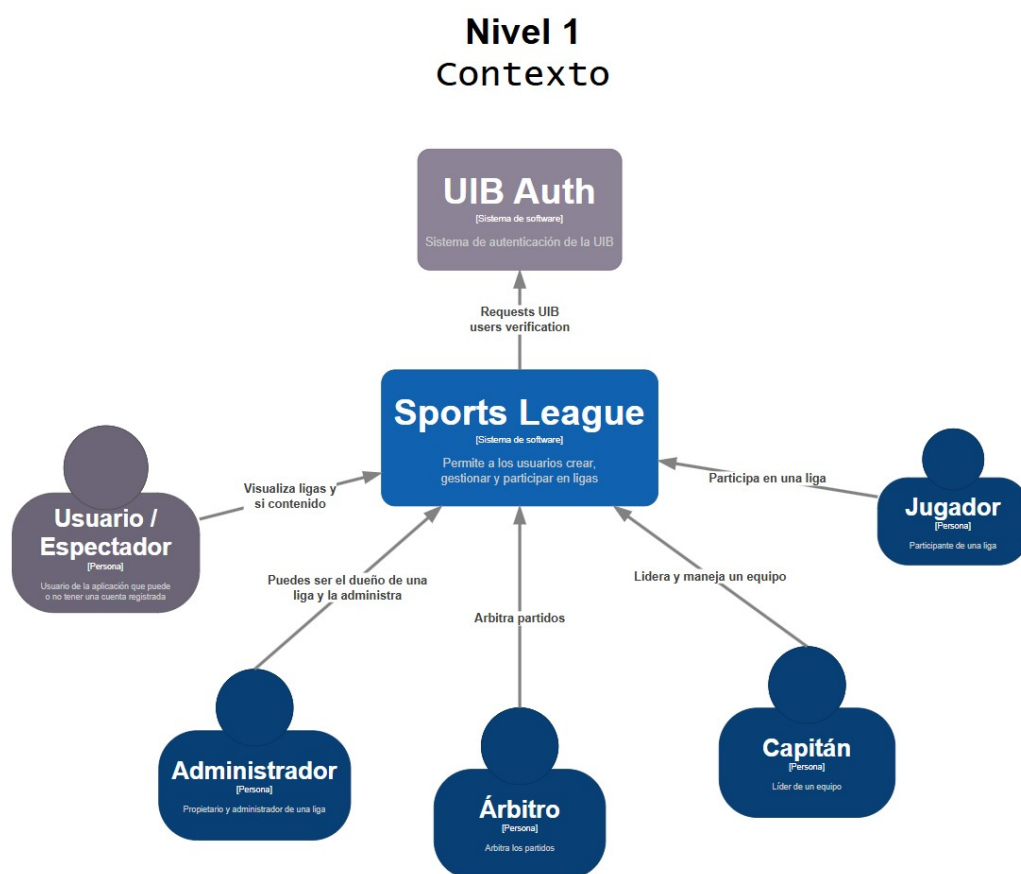


Figura I.1: Modelo C4 nivel 1

Nivel 2 Contenedores

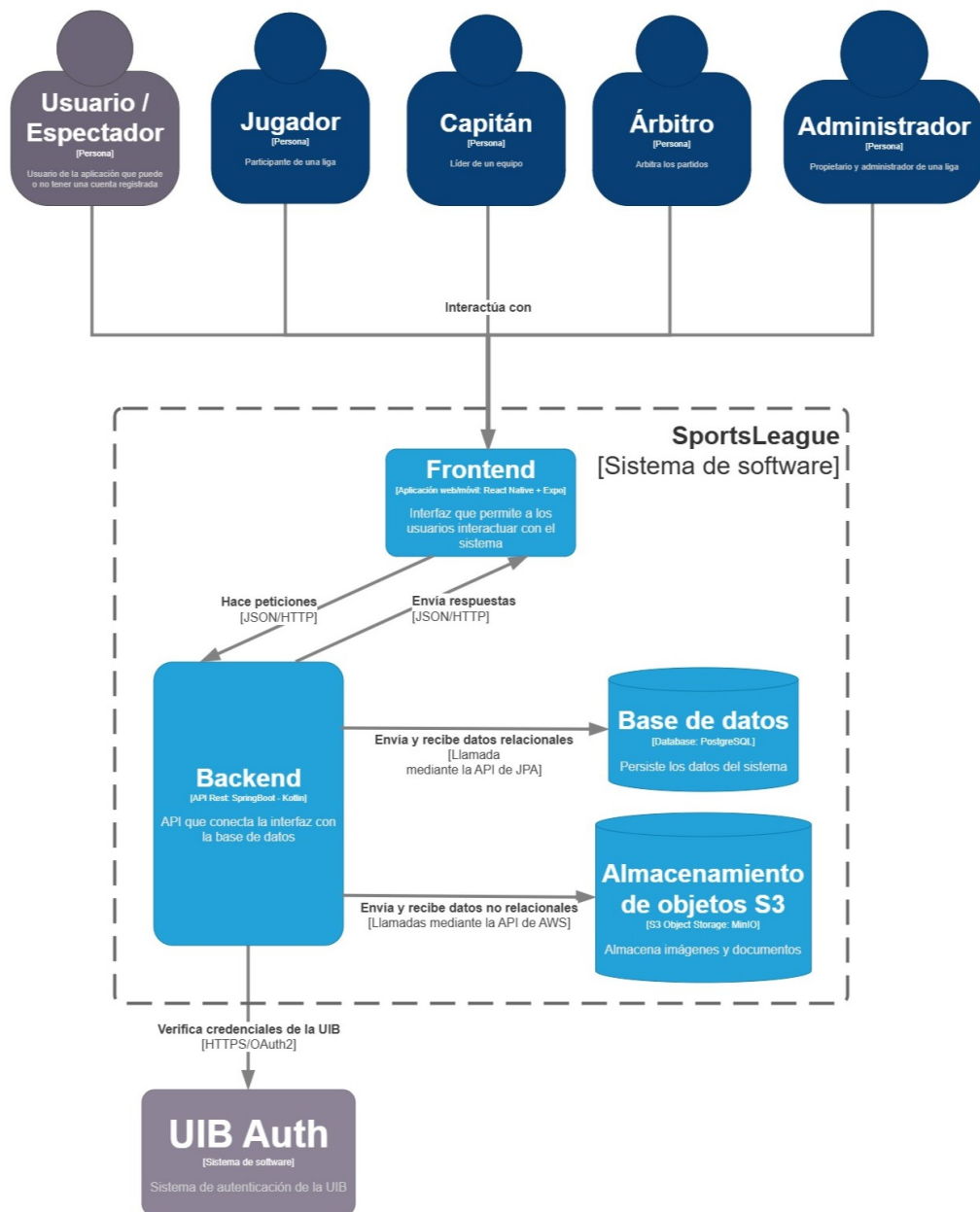
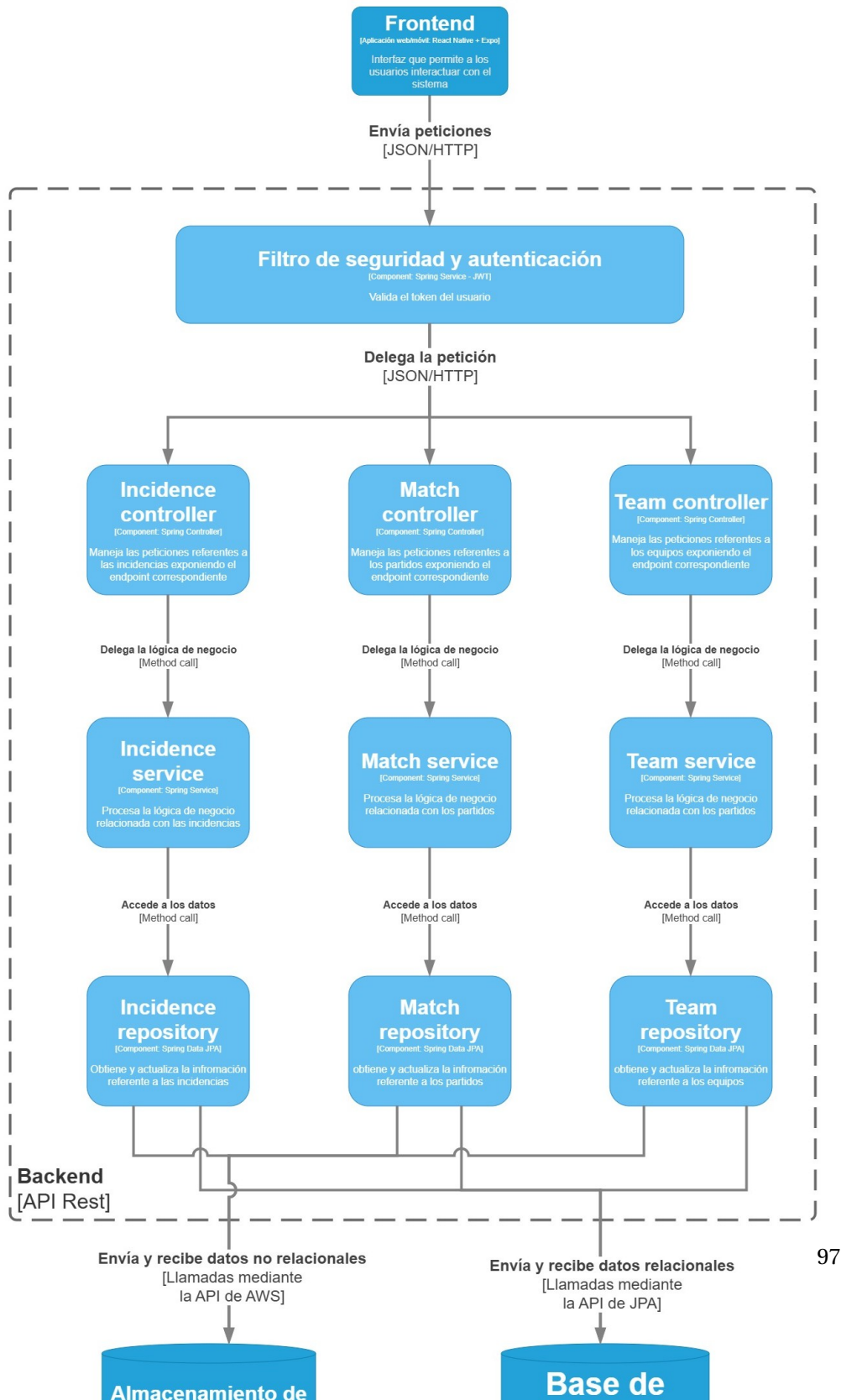


Figura I.2: Modelo C4 nivel 2

Nivel 3

Componentes: Backend (Modelo 3 capas)



I. DISEÑO DEL SISTEMA

Nivel 3 Componentes: Backend (Modelo hexagonal)

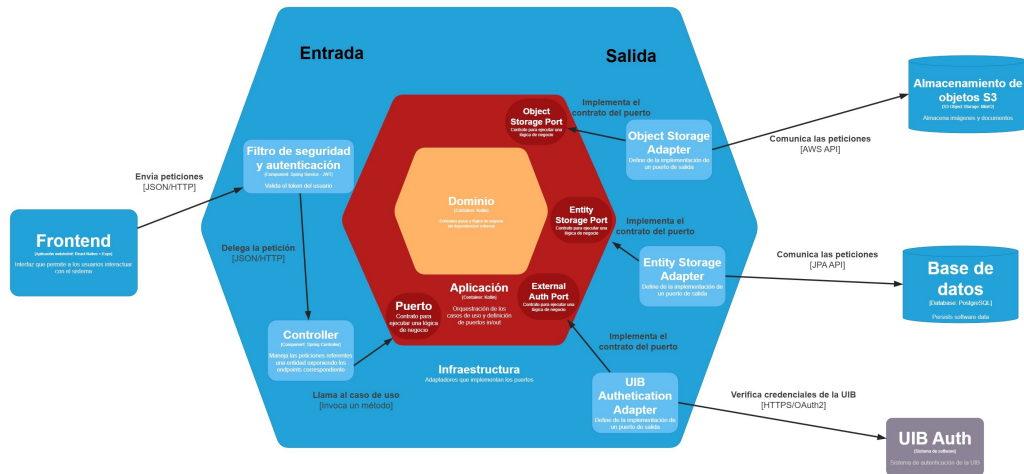


Figura I.4: Modelo C4 nivel 3: arquitectura hexagonal

Escenario de caso de uso: Inicio de sesión

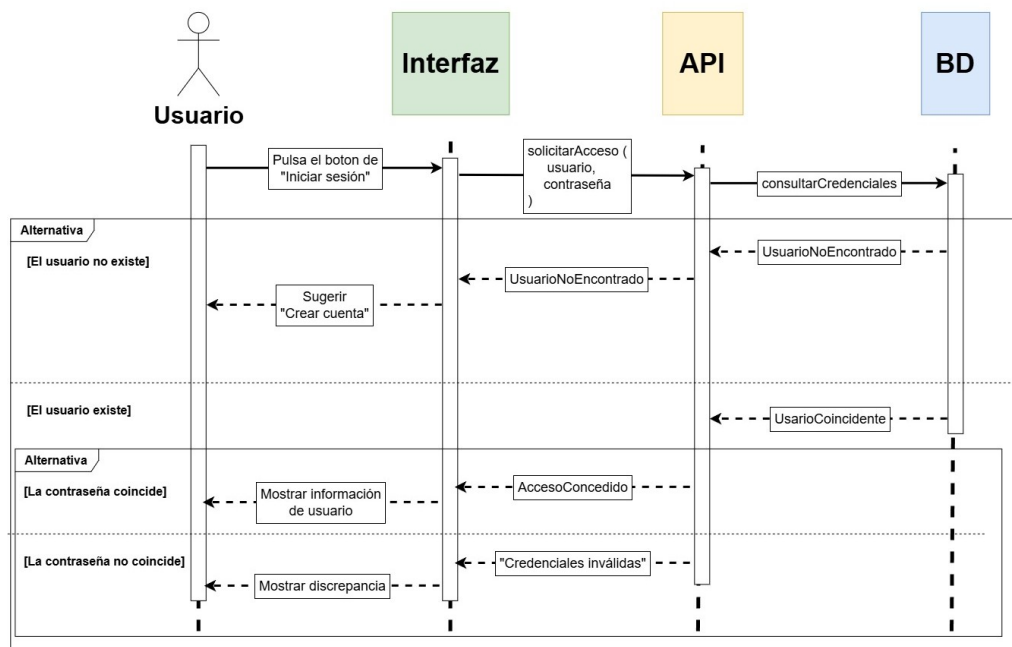


Figura I.5: Diagrama Usuario-Sistema para el caso de uso de Iniciar Sesión

Escenario de caso de uso:
**Generación de
enfrentamientos**

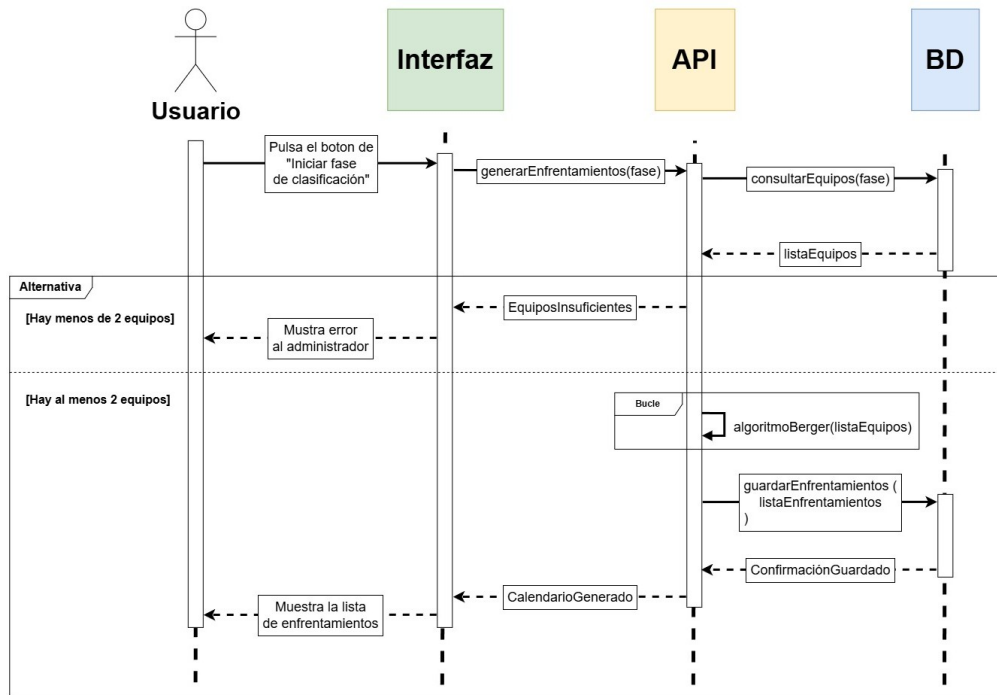


Figura I.6: Diagrama Usuario-Sistema para el caso de uso de Crear Enfrentamientos

BIBLIOGRAFÍA

- [1] Talent Apps, “Winner - Organizar torneos,” <https://play.google.com/store/apps/datasafety?id=il.talent.winner>, 2019, consultado: 2026-08-03. 1.1
- [2] Clupik, “Clupik - Software para clubes, academias y ligas,” Página web principal: <https://clupik.com/>, 2022, aplicaciones móviles disponibles: App para usuarios https://play.google.com/store/search?q=clupik&c=apps&androidId=111191616697929294962&utm_source=emea_Med. App para administradores https://play.google.com/store/apps/details?id=com.mileyenda.manager&utm_source=emea_Med. App para árbitros https://play.google.com/store/apps/details?id=com.mileyendareferee.app&utm_source=emea_Med. Consultado: 2026-08-03. 1.1
- [3] Robert C. Martin, “Design Principles and Design Patterns,” https://objectmentor.com/resources/articles/Principles_and_Patterns.pdf, 2009, consultado: 2026-08-25. 1.3
- [4] R. Likert, “A technique for the measurement of attitudes,” *Archives of Psychology*, vol. 22, no. 140, p. 55, 1932. 3.1.2
- [5] U. Itai, “Sports scheduling simplified: The power of the rotation algorithm in round-robin tournament,” <https://medium.com/coinmonks/sports-scheduling-simplified-the-power-of-the-rotation-algorithm-in-round-robin-tournament-eedfbd3fee9> 2024, consultado: 2026-09-03. 3.2.2
- [6] OpenAPI Tools, “Openapi generator gradle plugin,” <https://plugins.gradle.org/plugin/org.openapi.generator>, 2026, consultado: 2026-09-03. 5.1.4
- [7] Acacode, “swagger-typescript-api,” <https://www.npmjs.com/package/swagger-typescript-api>, 2026, consultado: 2026-09-03. 5.1.4
- [8] TanStack, “Postmortem: Tanstack npm supply-chain compromise,” <https://tanstack.com/blog/npm-supply-chain-compromise-postmortem>, 5 2026, consultado: 2026-09-01. 6.2.1